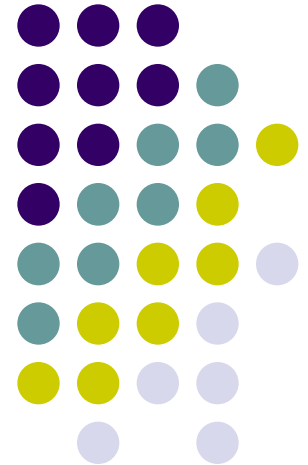
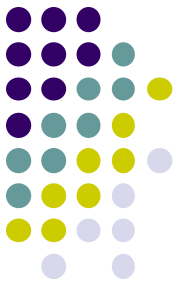


Natural Language Processing

CPSC 488

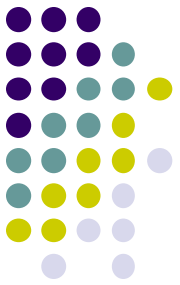
Word Embeddings





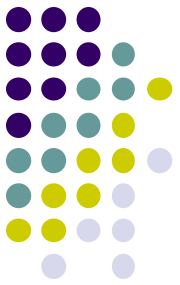
Lecture Overview

- Concepts of self-supervised learning
- Word2Vec word embeddings
 - Continuous Bag of Words (CBOW)
 - Skip-gram
- Analogical reasoning with embeddings
- Bias in embeddings
- Applications of embeddings



Learning Word Embeddings

Embeddings



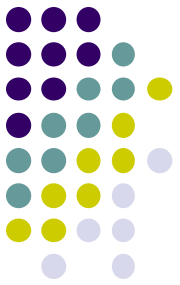
- **What is Embedding?**

- A means of **representing objects** like words, sentences, images, audios, users, etc. as **points in a continuous vector space**.
 - In mathematics, one instance of mathematical structure contained within another instance
- Different from the Term-Context Matrix

- **Examples**

- **Word embeddings**
 - Represent words as context vectors
- **Sentence/document embeddings**
 - Capture the meaning of sentences or documents
- **Image embeddings**
 - Convert images into vectors for recognition
- **User/item embeddings**
 - Represent users and items numerically to find matches in recommendation systems (movies, products, music, etc.)
- **Graph/network embeddings**
 - Represent nodes in a network (social media) as vectors

Example Word Embedding

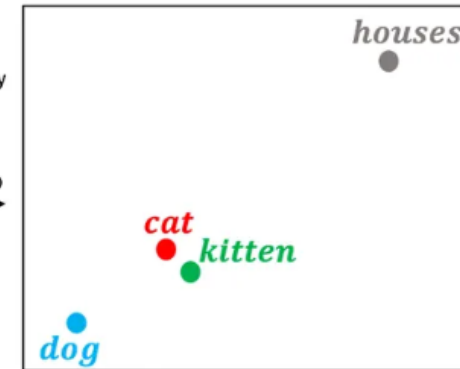


dimension

	living being	feline	human	gender	royalty	verb	plural
<i>cat</i> →	0.6	0.9	0.1	0.4	-0.7	-0.3	-0.2
<i>kitten</i> →	0.5	0.8	-0.1	0.2	-0.6	-0.5	-0.1
<i>dog</i> →	0.7	-0.1	0.4	0.3	-0.4	-0.1	-0.3
<i>houses</i> →	-0.8	-0.4	-0.5	0.1	-0.9	0.3	0.8

Dimensionality
reduction of
word
embeddings
from 7D to 2D

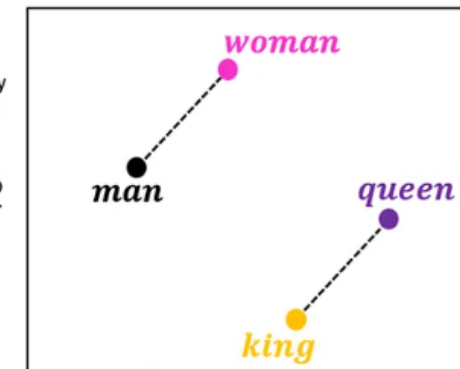
Words in continuous vector space



Vectors

<i>man</i> →	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7
<i>woman</i> →	0.7	0.3	0.9	-0.7	0.1	-0.5	-0.4
<i>king</i> →	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6
<i>queen</i> →	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9

Dimensionality
reduction of
word
embeddings
from 7D to 2D



Word2Vec Word Embeddings

(Mikolov et al., 2013)



- **Intuition:**

- **Context** is the *information* that surrounds a word, phrase, sentence, or document that helps clarify its meaning and purpose.
 - Semantically related words are likely to occur near each other in similar contexts and lead to similar word embeddings.
 - Words used in a specific context that can change their meaning

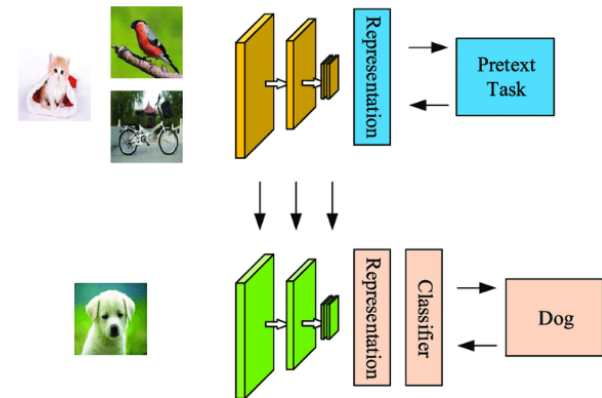
- **How to get the embeddings**

- **Predict context words** for a given **target word** or **vice versa**
 - The output will be a **dense embedding vector** that captures the relationships.
 - (The idea comes from **neural language modeling** by Bengio et al., 2003; Collobert et al., 2011)
- **Comparison to N-gram LM**
 - Similar, but predicting the next word is **not the only final goal** (but a dense vector)

- **But do we have training data?**

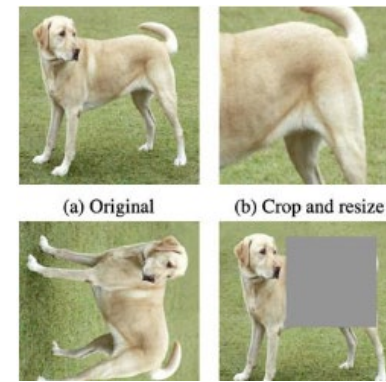
- We don't usually have the typical training dataset with labels but...

Self-Supervised Learning



- **Idea:**
 - Use the **labels** that are **naturally** part of the **(unlabeled)** data.
- **Steps in Self-Supervised Learning (SSL)**
 1. **Generate supervisory signals** from **unlabeled data** by **designing a pretext tasks** (e.g., predicting missing or masked parts, **contrastive learning**).
 - **Pretext task**: a task designed to force the model to learn features.
 2. *Train the model* based on the pretext task using supervised learning to learn meaningful feature representations.
 3. *Use the learned representation and fine-tune* them on downstream tasks (actual tasks for target applications, e.g., sentiment analysis, object detection)
- **Examples**
 - Running text as training data
 - E.g., for a sentence, “**The cat sat on the mat.**”, randomly **mask** a word “The cat [] on the mat.”, then train the model to predict the masked word “[**sat**]”.
 - Computer vision
 - Predicting missing patches of an image, object detection, image generation

Self-Supervised Learning (SSL)



- **Capabilities of SSL**

- Predict **any part** of the input **from any other part**
 - Pretend there is a part of the input you don't know and predict that.
- Predict the **obscure** from the **visible vice versa**
- Predict the **future** from the **past** or the **past** from the **present**

- **Connections with other ML approaches**

- Supervised learning, Unsupervised learning, Contrastive learning
- Semi-supervised learning
- Reinforcement learning

- **Benefits**

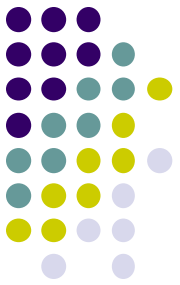
- **Scalability** – without expensive training data
- **Understanding** and **mimicking** how the human mind works
 - Machine's capability of automatically generating labels without any humans

- **Limitations**

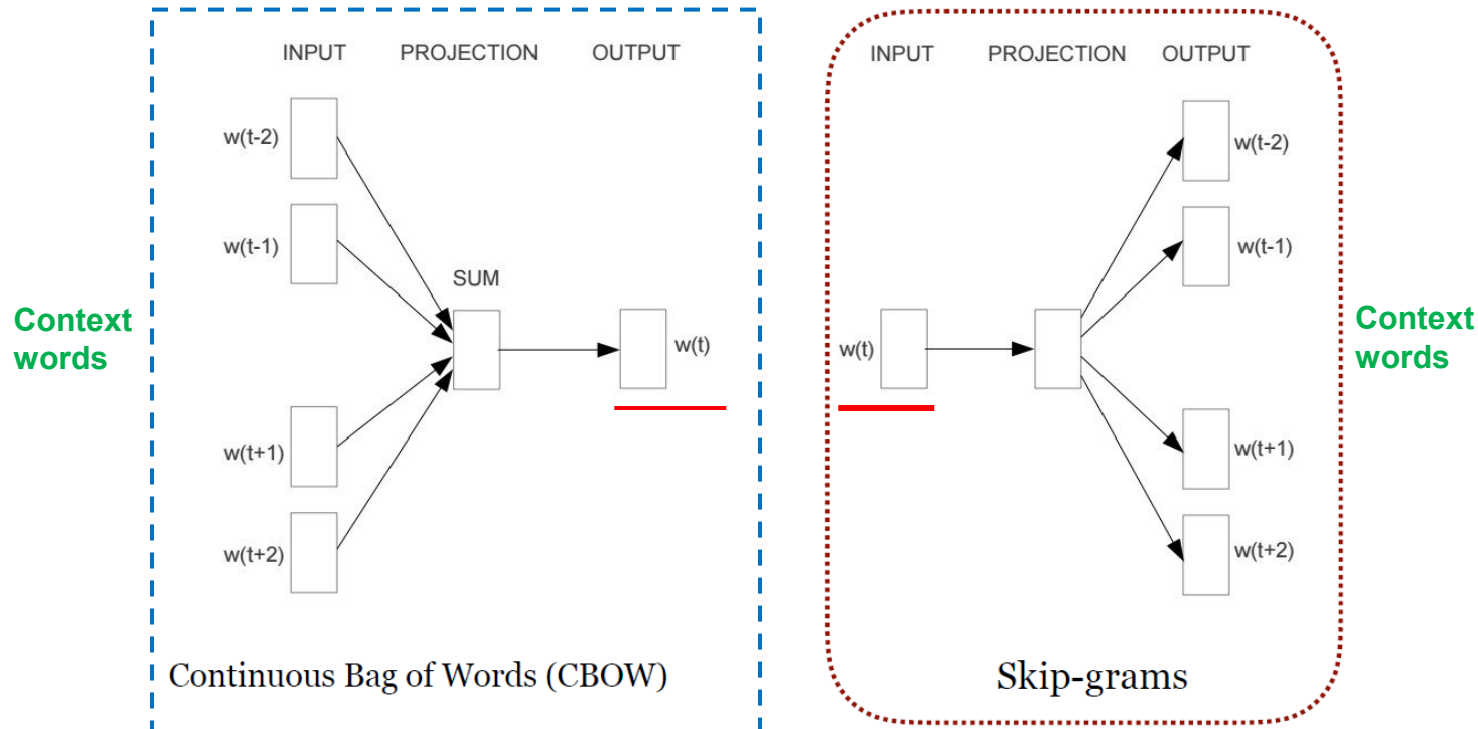
- Requires a lot of computational power
- May be low accuracy

Word2vec Word Embeddings

(Mikolov et al., 2013)



- **Word2vec comes in two main models:**
 - Continuous Bag of Words (CBOW)
 - **Input data are not frequencies.** Why is it called **continuous bag of words**?
 - Skip-gram



Continuous Bag of Words (CBOW)



- **Network architecture**

- Input layer
 - The **context words**, each represented as **one-hot encoded vector**.
 - The window size is determined by # of context words.
- Hidden layer
 - The input one-hot encoded vectors are **combined** (by average or sum).
- Output layer
 - The output from the hidden layer is fed into a **Softmax layer** for a **probability distribution** that **predicts the target word**.

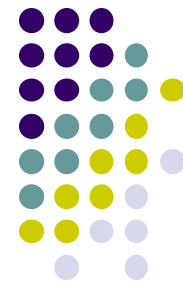
- **Training**

- Adjust the weights to **maximize the probability** of the target word, given the context words.
 - Maximize $P(w_t|context) = P(w_t|w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2},)$

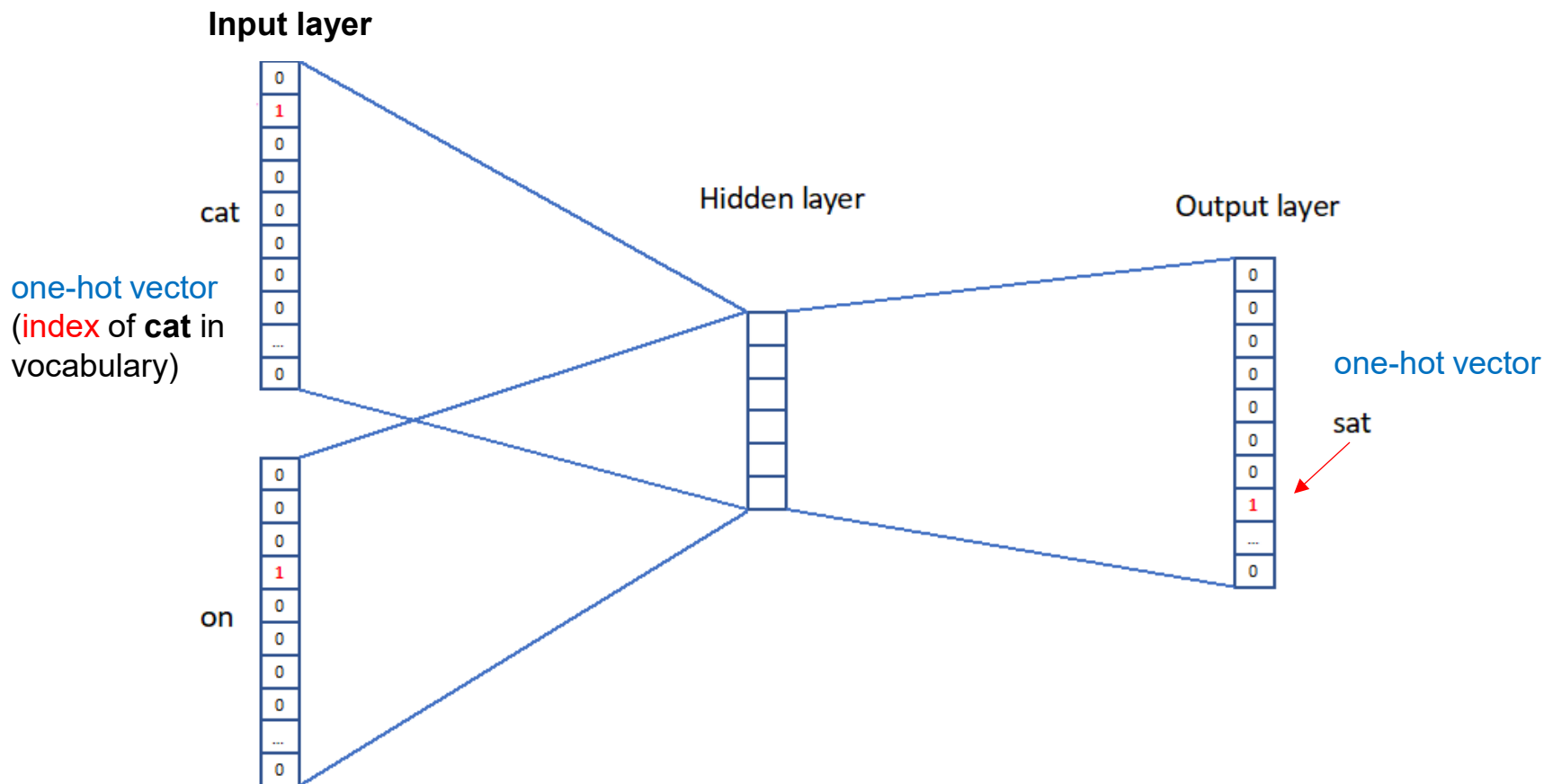
- **Word embeddings**

- The **weights** that capture meaningful semantic relationships between words).

CBOW: A Shallow Neural Architecture



“The cat [sat] on the mat.” Assume window size = 2

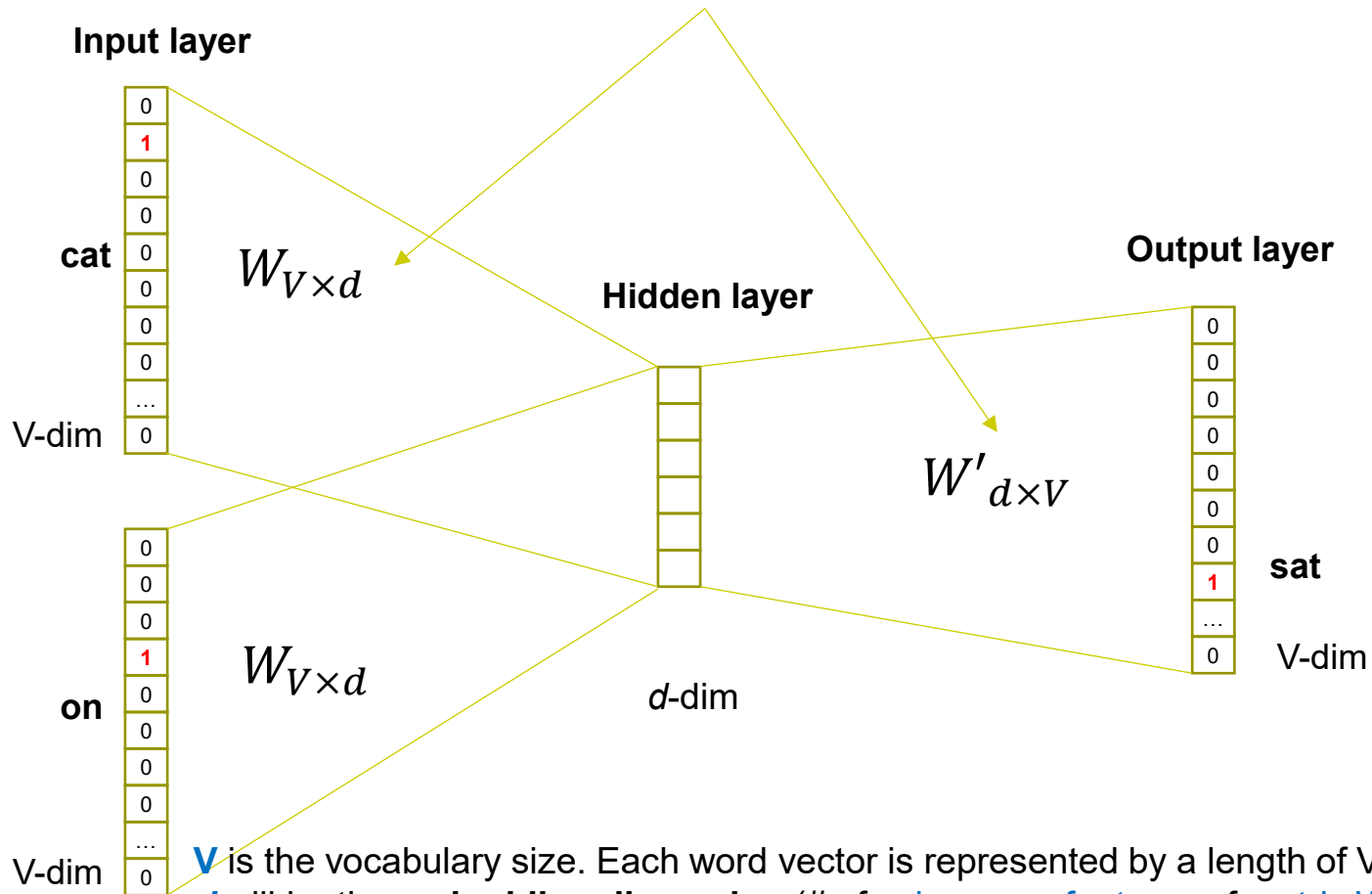


A shallow neural network with one hidden layer to learn word embeddings



CBOW to Learn Word Embeddings

We must learn weight matrices W and W'



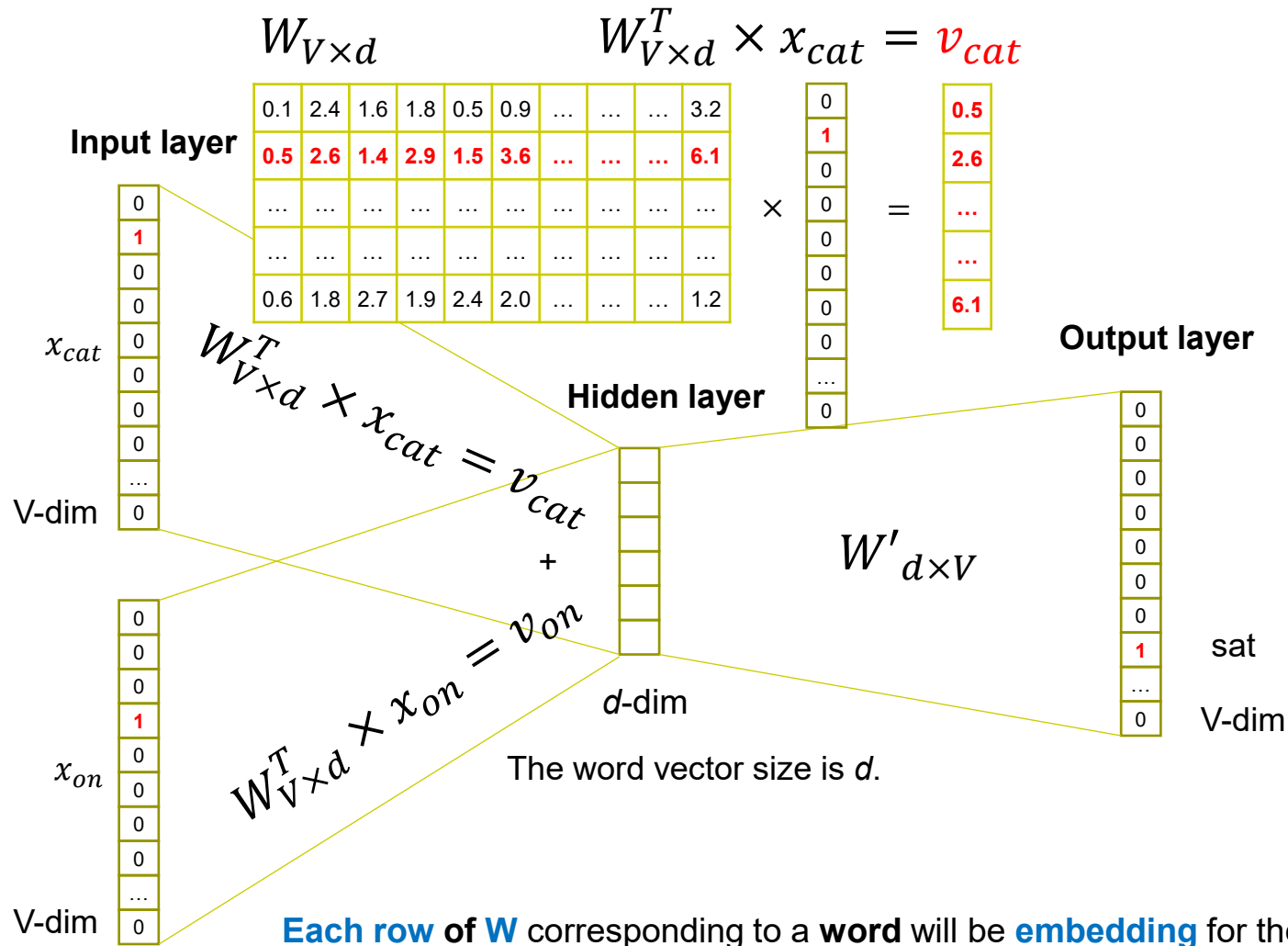
V is the vocabulary size. Each word vector is represented by a length of V , $|V|$.

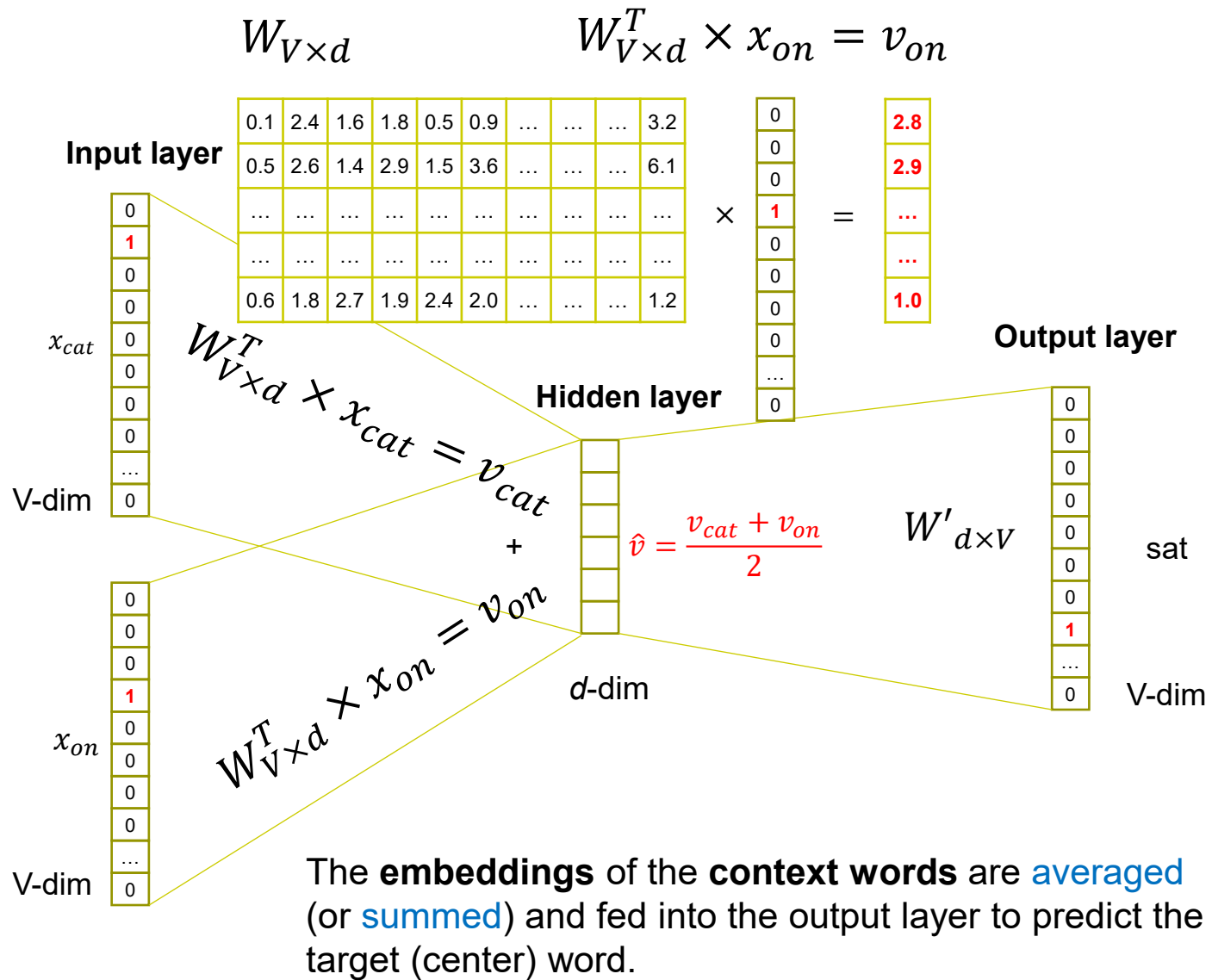
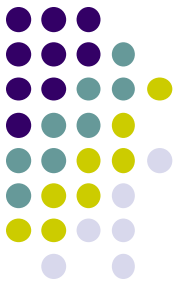
d will be the **embedding dimension** (# of columns or features of matrix W).

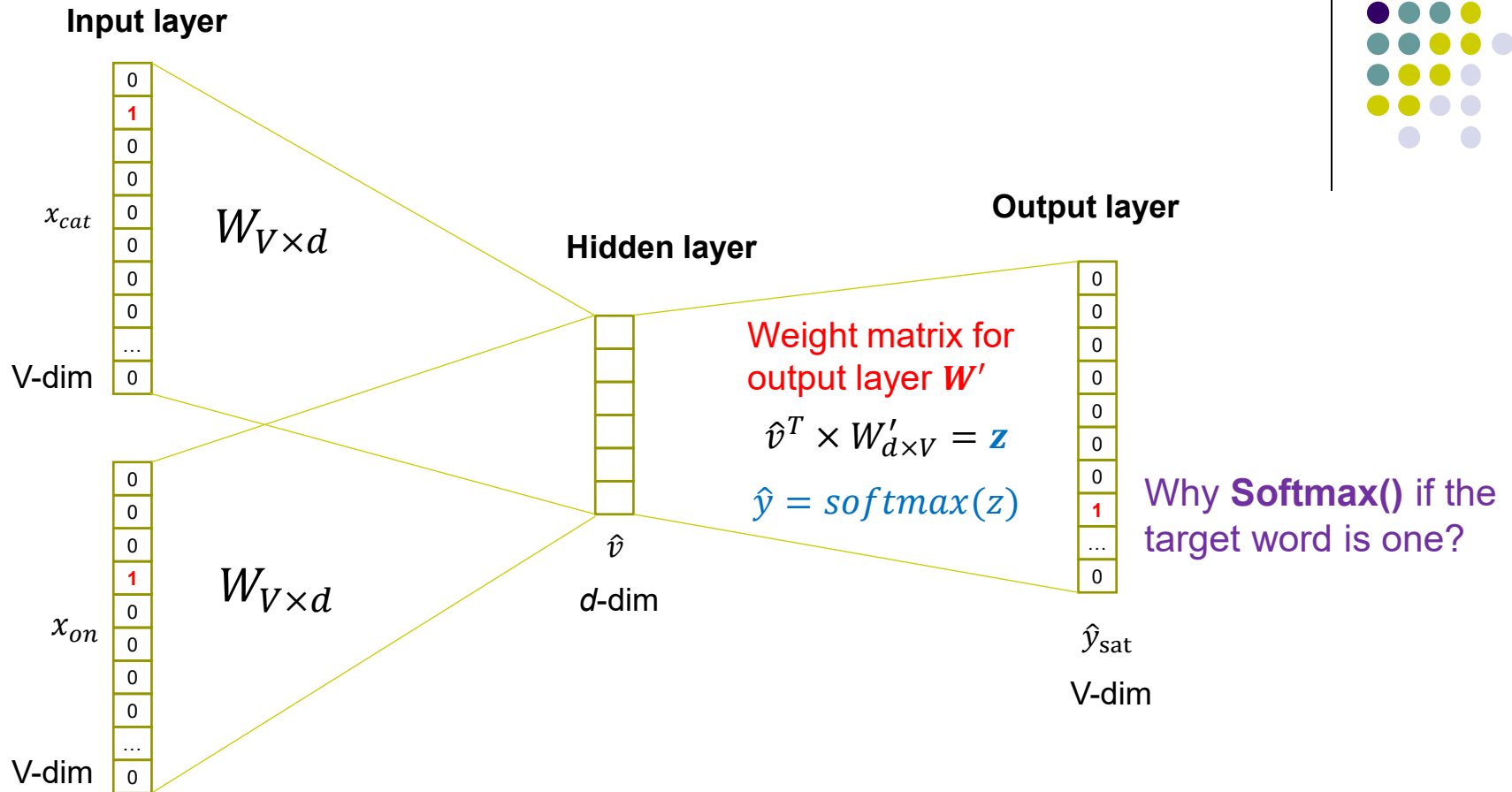
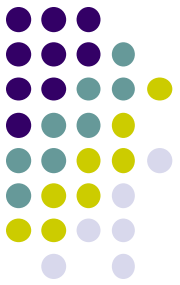
What are features? Each dimension is a distributed feature that encodes multiple aspects of the contexts a word appear in but a compressed representation.



To extract an embedding vector, multiply **W** by one-hot vector.

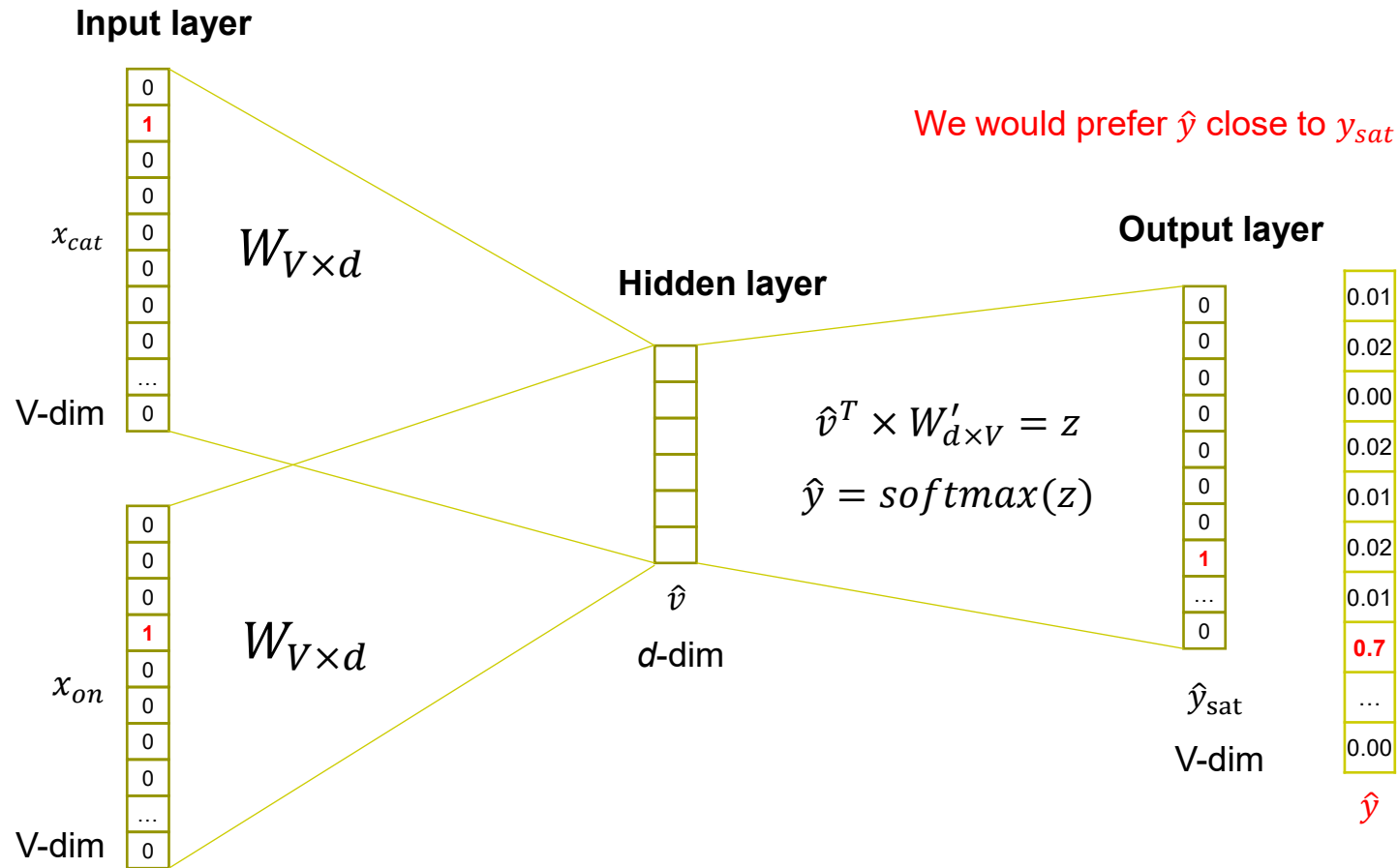
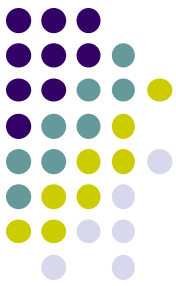






Activation functions: no activation function needed
(but ReLU or Tanh can be used for non-linearity at hidden layer).

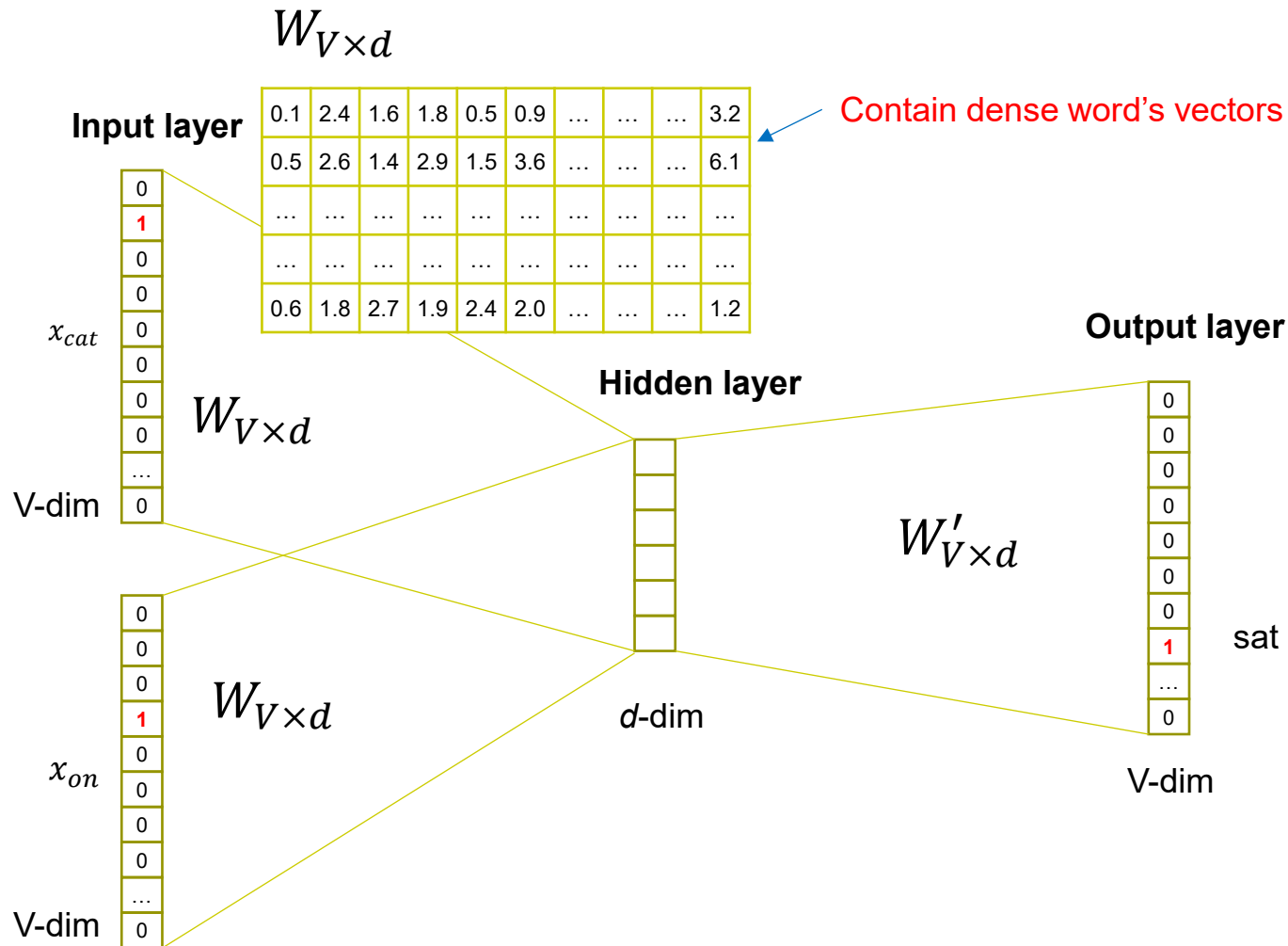
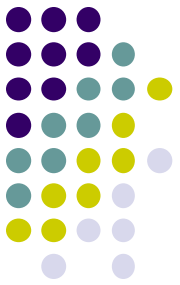
Output layer: $\text{Softmax}(z)$ computes $P(w_i | \text{context}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ where z_i is the score.



Backpropagation: Given a set of context words C and target word w_i (“sat” in this example), the objective is to maximize the probability $P(w_i|context)$ or $P(sat|context)$.

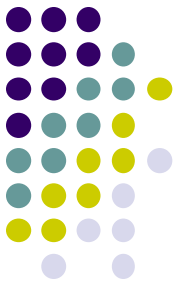
Loss function using cross-entropy: $-\sum_i y_i \log P(w_i|C)$ from $-\sum_i y_i \log \hat{y}$
 $-\sum_i \log P(sat|C)$ (0.7 is only for target word “sat”)

The weight matrices W and W' are updated by the gradient descent.



When the context words change ["sat", "the"], the target word is changed (shifted to) "on" from "the cat sat on the mat". **This process is repeated for an entire corpus.**

Which ones are embeddings? W (standard) or by taking average



How to Deal with Large Vocabularies

- **The huge text corpora**

- Wikipedia > 6B (full version >100GB), Twitter > 27B, Google documents, etc.

- **Idea 1: Stochastic Gradient Descent (SGD)**

- Still take too much time as it still calculates probabilities for every word

- **Idea 2: Negative sampling** (Morin and Bengio 2005)

- **Positive examples**

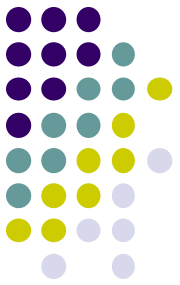
- Examples with actual target word and context words

- **Negative examples**

- Randomly samples with a few target words which are unrelated to the context
- Serve as noise

- **Idea 3: Hierarchical Softmax**

- Softmax $P(w_i|h) = \frac{e^{h^T w_i}}{\sum_j e^{h^T w_j}}$ (h is the hidden state and w_i is the weight vector for word w_i .)
 - The denominator sums **over all words** (too many, very expensive)
 - Uses a **binary tree structure** to break down the process into multiple binary decisions (Morin and Bengio 2005).



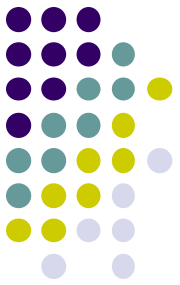
Positive and Negative Examples

c_1 c_1 *target* c_1 c_1
...lemon, a [tablespoon of apricot jam, a] pinch...

- Positive examples
 - Every (or frequent) pair of (target, context) within the context window
- Negative examples
 - For each positive example, we'll grab **k negative examples** (with words **not** in **context words**), by sampling or choosing noise words.

positive examples +	
t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -			
t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if



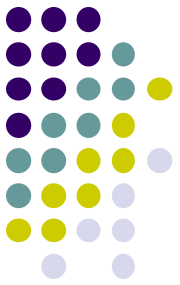
Choosing Negative Examples

- **Frequency-based sampling**

- Pick w according to their unigram frequency $P(w)$
 - More frequent words have a higher chance of being selected.

- **Sampling by probability distribution**

- More commonly, chose according to $P_\alpha(w_i) = \frac{\text{count}(w_i)^\alpha}{\sum_j \text{count}(w_j)^\alpha}$
- $\alpha = 3/4$ (**0.75**) **works well** because it gives **rare noise words** slightly higher probability.
 - To show this, imagine two events $P(a) = .99$ and $P(b) = .01$
 - $P_\alpha(a) = \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = 0.97$
 - $P_\alpha(b) = \frac{.01^{.75}}{.99^{.75} + .01^{.75}} = 0.03$



Why Negative Sampling?

- **Computational and memory efficiency**

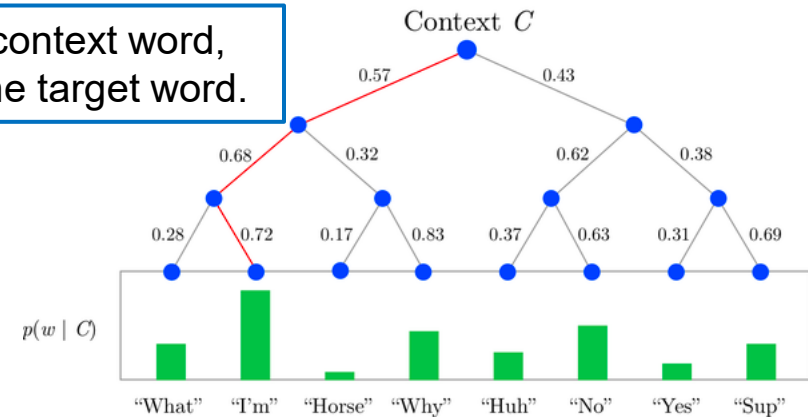
- Instead of predicting the entire vocabulary, update weights only for a small number of negative samples to speed up training time.
- Computational efficiency
 - The normalization term at Softmax layer can very expensive (with many classes)
 - $$P(t_w|c_w) = \frac{\exp(u_t^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$
- Improved training efficiency from fast convergence => scalability

- **Better representation of word similarity and rare words**

- By contrasting the real world-context pairs with incorrect samples (called “contrastive learning”), the model is able to learn better.
- Improved embedding quality => good performance

Given a context word,
predict the target word.

Hierarchical Softmax

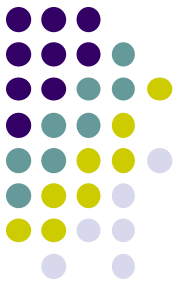


● Computing the probability

1. Construct a (balanced) binary tree (in advance)
 - We want **related** (similar) **words** close to each other.
 - **Huffman encoding** (tree) that minimizes the length, shorter codes for frequent symbols, longer codes for less frequent ones (used in data compression).
2. Traverse the tree by binary decision
3. Compute the probabilities at each intermediate node n ($n = |V| - 1$ in Huffman)
 - $\sigma(z_n) = \frac{1}{1+e^{-z_n}}$ where $z_n = \mathbf{h}^T \mathbf{v}_n$ ($\mathbf{h}$ is the hidden vector of the context word C).
 - $\mathbf{W}'$ is replaced by parameter vectors $\mathbf{v}_n$ at internal nodes.
4. Compute the final probability by multiplying all the probabilities at each node on the path at the leaf node.

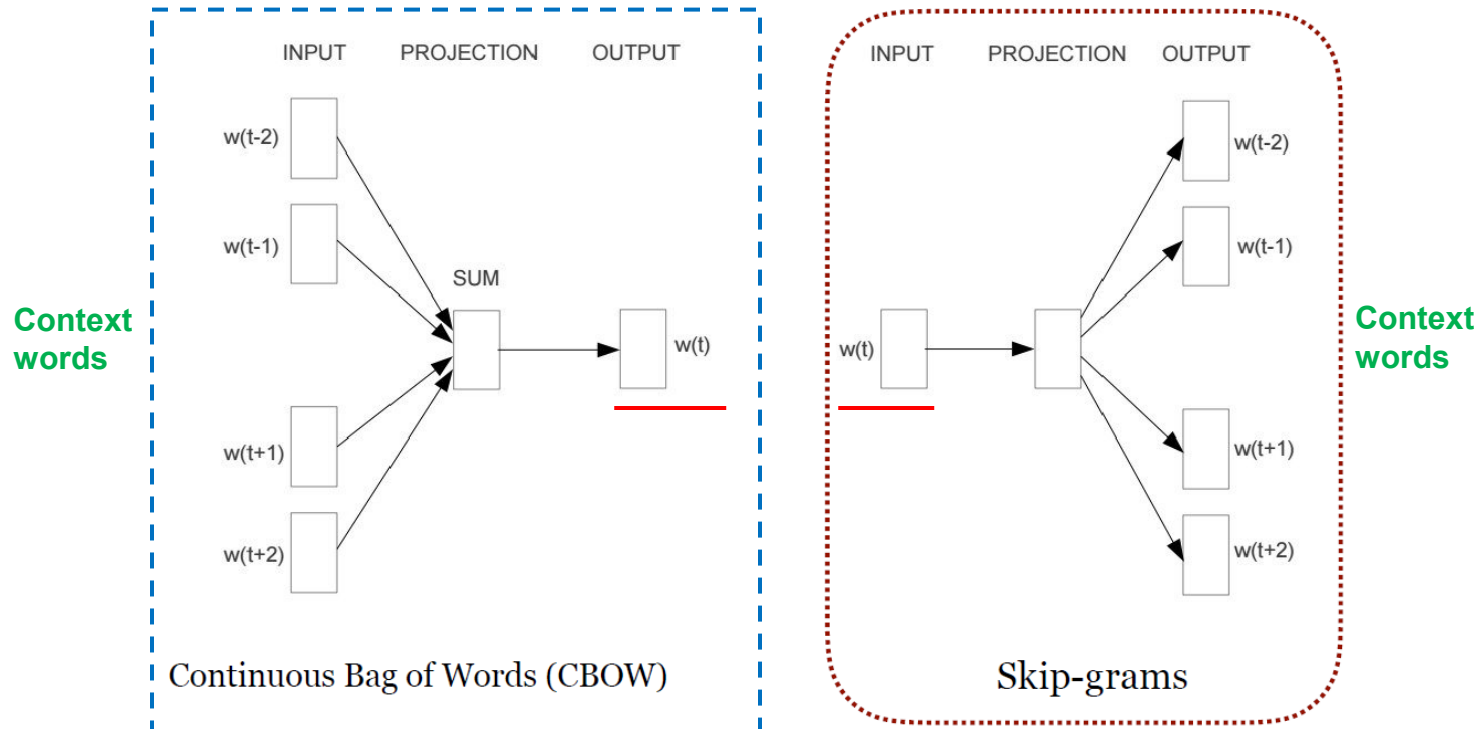
● Updating weights via backpropagation

- Forward pass => Loss calculation
- Backward pass to update the **weights** $\mathbf{W}$ at **each intermediate node**.

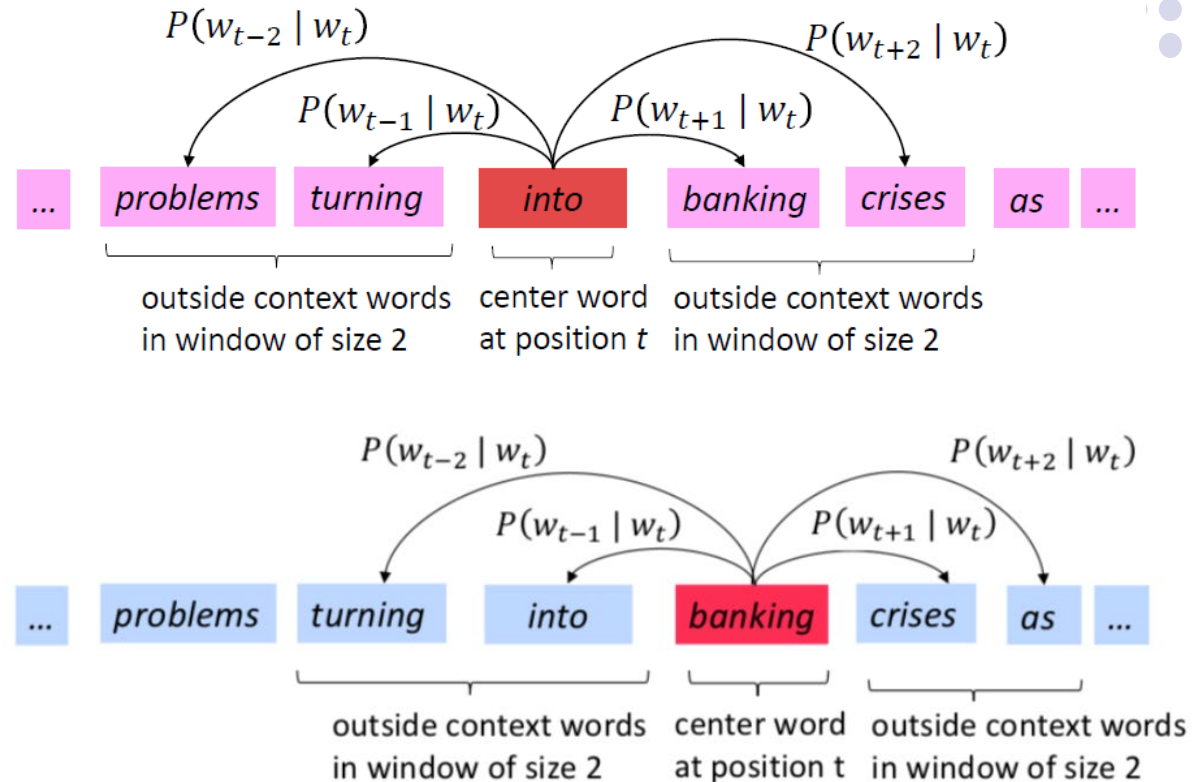
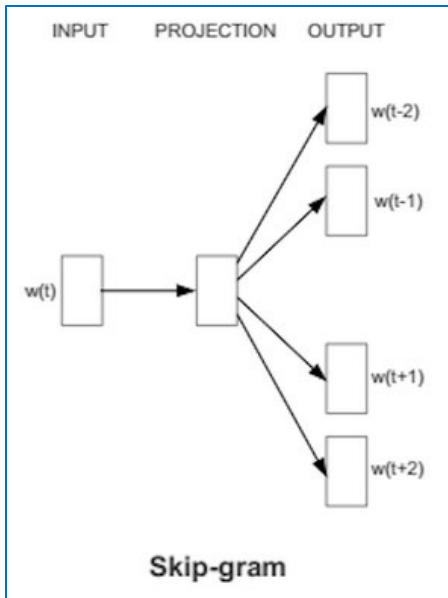


Skip-Gram

- Word2vec comes in two main models:
 - Continuous Bag of Words (CBOW)
 - **Skip-gram**



Skip-gram Idea



- Start with a **target word** and **predict** the **context words**.
 - Compute the probability for **each pair** (target, context) surround words within the window size.

Skip-gram



- **Network architecture**

- Input layer
 - One-hot encoded vector representing the target word
- Hidden layer
 - Each word is projected into a continuous vector space (vector).
 - The weights between input and hidden layer will be the learned embeddings.
 - No activation function needed
- Output (Softmax) layer
 - Computes the probabilities of all words in V being the context word.

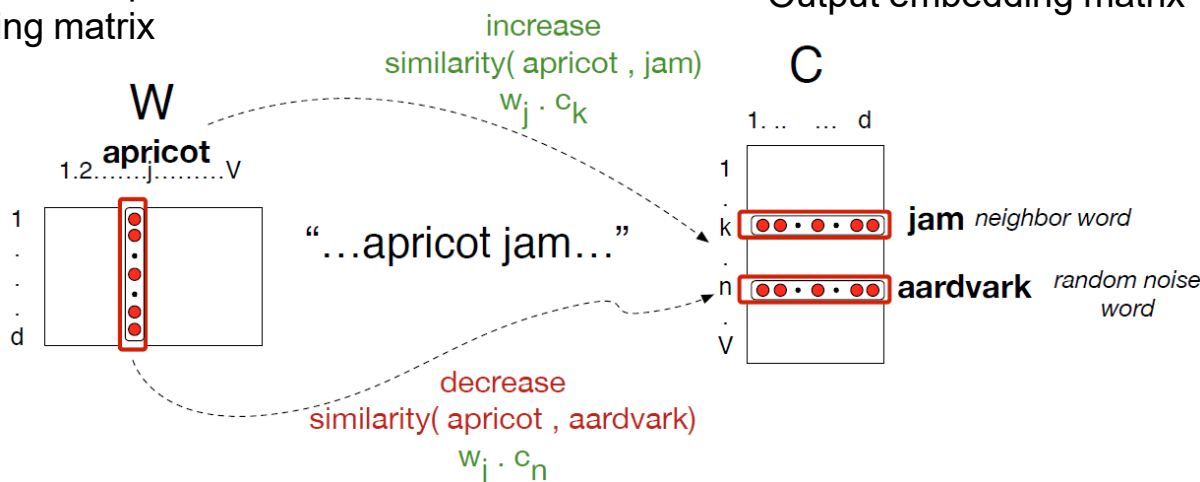
- **Training**

- First, two embedding matrices are initialized randomly.
 - Input embedding matrix W ($V \times d$) for $|V|$ target words
 - Output embedding matrix C (W') ($d \times V$) for $|V|$ context words
- Adjust these embedding matrices to maximize $P(w_{context} | w_{target})$ via the gradient descent.

Context Scores

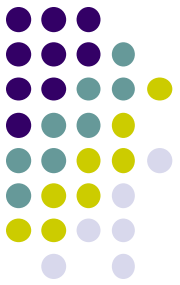


d dimensional input
embedding matrix



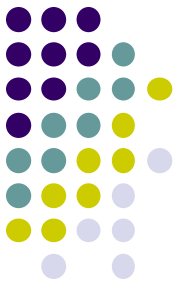
● Calculating context word scores

- Each column of C represents the embedding vector for a context word.
- For each context word c_j , the score $z_j = w_{target} \cdot c_j$
 - w_{target} and c_j are the target and context word's embedding vectors.
 - The score can be similarity measure using dot product or Cosine function.
- These scores z_j are passed to Softmax layer to get the probability distribution over all possible context words.



From Similarities to Probabilities

- Skip-gram assumes that all context words are independent for simplification.
 - $P(+|w, c_1:L) = \prod_i^L \sigma(c_i \cdot w)$
 - $P(+|w, c_1:L) = \sum_i^L \log \sigma(c_i \cdot w)$
- **Dot product to compute similarity**
 - Two vectors are similar if they have a high dot product.
 - Cosine is just a normalized dot product.
 - So, $\text{Similarity}(w, c) \approx w \cdot c$
- To turn this into a probability, pass it to the multinomial $\text{Softmax}(z)$



Learning Embedding Matrix W

- **Objective function for negative sampling:**
 - (Binary) **Logistic loss**, instead of **cross-entropy** (for computational efficiency)
$$J = \log(\sigma(v_{w_t} \cdot v_c)) + \sum_i^k \log(\sigma(-v_{w_i} \cdot v_c))$$
 where $\sigma(x)$ is the logistic function.
 - v_{w_t} is the embedding of the true target word.
 - v_c is the aggregated embedding of the context words.
 - v_{w_i} is the embedding of the i-th negative sample.
 - k is the number of negative samples (typically $k = 5 - 20$).
 - Increasing k typically improve the quality but take more time.
- **Maximize** for **positive (true) examples** but **minimize** for **negative examples**.



Loss Function with *Negative Sampling*

- **Deriving Cross-entropy (CE) loss**

from objective function $\sum_{(t,c) \in +} \log P(+|t, c) + \sum_{(t,c) \in -} \log P(-|t, c)$

$$L_{CE} = -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right]$$

since each positive pair with **k** noise words

$$= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right]$$

each term is cross-entropy since the true label y is one-hot encoded, **y=1**

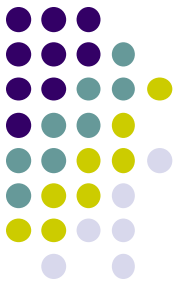
$$= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right]$$

$$= -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

converting to probabilities using **sigmoid**

- **Training objective using negative sampling**

- **Maximize** the similarity (target, context) pairs drawn from the positive examples but **Minimize** the similarity (target, context) pairs drawn from the negative examples.
 - By Stochastic Gradient Descent based on negative sampling



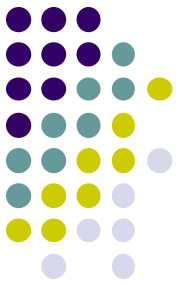
The Derivatives of the Loss Function

$$L_{CE} = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$



Update Rule in SGD

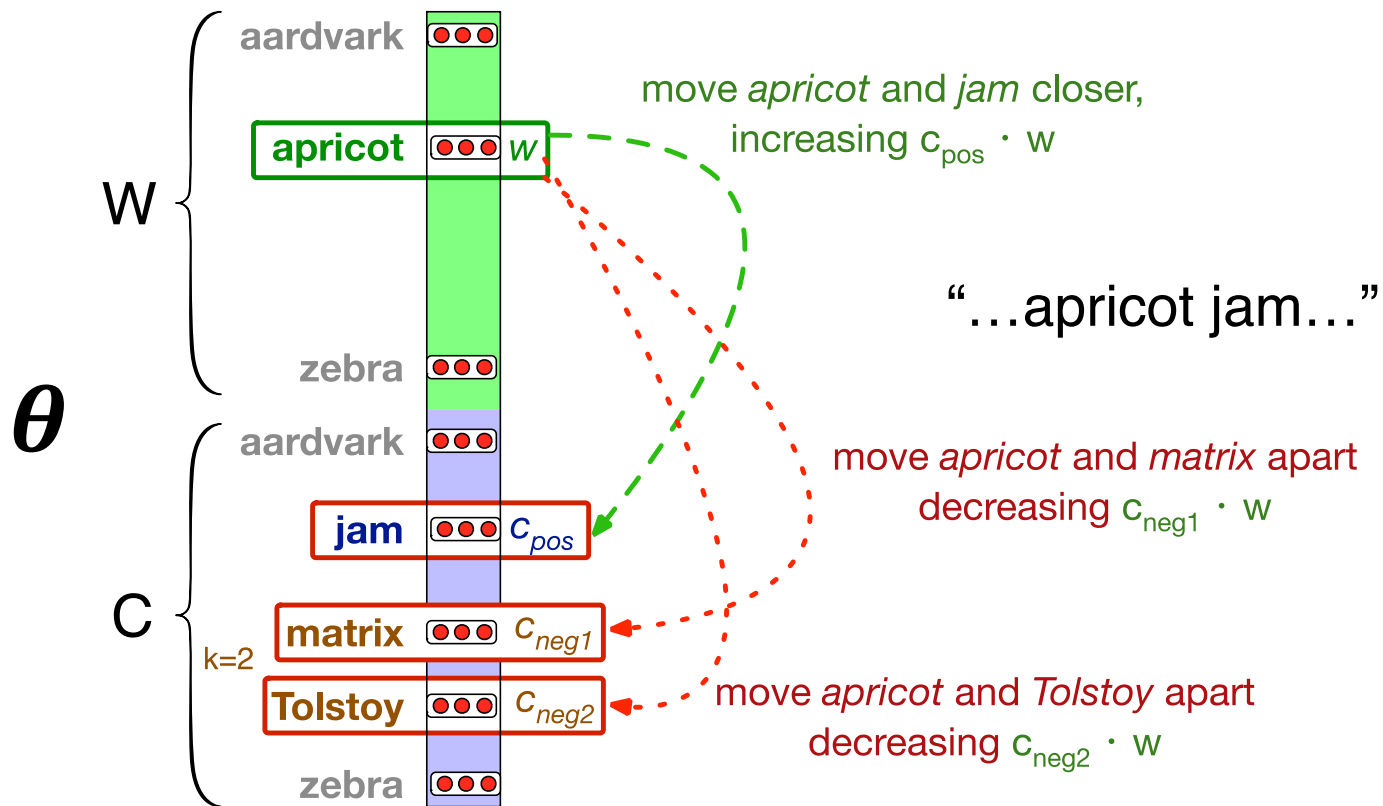
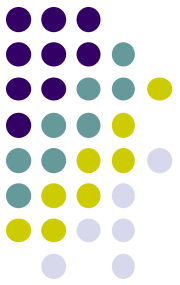
Start with randomly initialized **C** and **W** matrices, then incrementally updates them.

$$c_{pos}^{t+1} = c_{pos}^t - \eta [\sigma(c_{pos}^t \cdot w^t) - 1] w^t$$

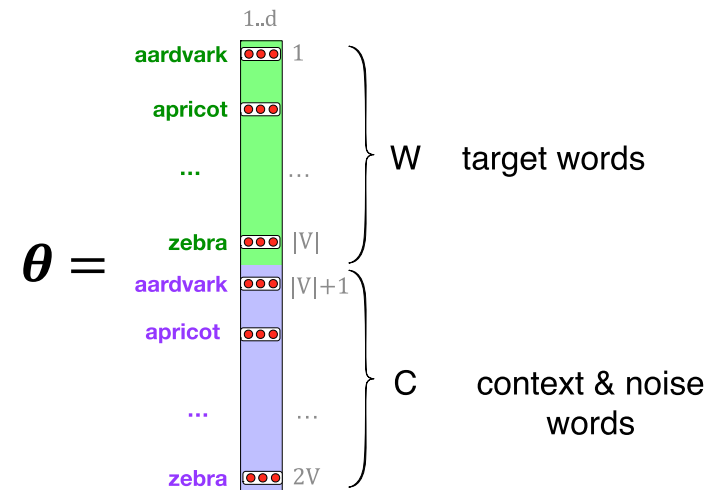
$$c_{neg}^{t+1} = c_{neg}^t - \eta [\sigma(c_{neg}^t \cdot w^t)] w^t$$

$$w^{t+1} = w^t - \eta \left[[\sigma(c_{pos} \cdot w^t) - 1] c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w^t)] c_{neg_i} \right]$$

Intuition of one step of gradient descent



Two Sets of Embeddings

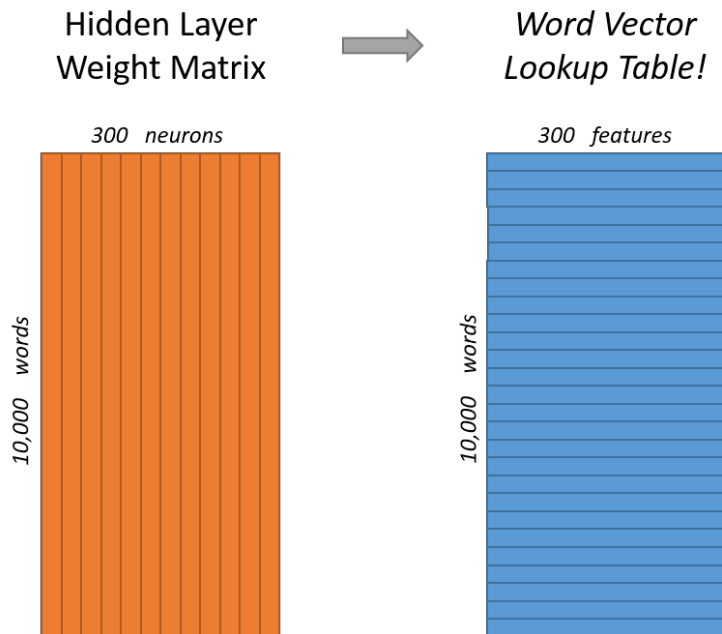


- **Skip-gram learns two sets of embeddings:**
 - Target embeddings matrix W
 - Context embedding matrix C
- It's common to just **add them together**, representing word i as the vector $w_i + c_i$
- To extract the embedding from the embedding matrix, use the **one-hot encoded vector** for the (target) word.



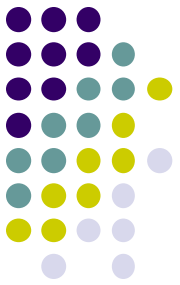
Example: Skip-gram Embedding Vector

- For vocabulary of 10,000 words ($V=10,000$)
- Embedding vectors with 300 features ($d = 300$ dimensions)
- So, the hidden layer is going to be represented by a weight matrix with 10,000 rows (multiply it by one-hot encoded vector on the left).



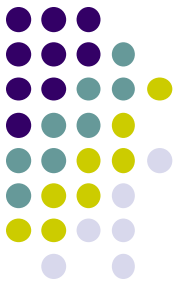
$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$

e.g., extracting embedding vector using one-hot encoded vector



Properties of Embeddings

- **Small windows ($C=\pm 2$):**
 - Nearest words are **syntactically similar words** in the **same taxonomy**
 - $C = \pm$ the nearest words to “**Hogwarts**”:
 - Sunnydale, Evernight, Blandings
- **Large windows ($C=\pm 5$):**
 - Nearest words are **related words** in the **same semantic field**
 - $C = \pm$ the nearest words to “**Hogwarts**”:
 - Dumbledore, Malfoy, Halfblood
- Similarity depends on window size of C .



CBOW vs Skip-gram

Key differences

Feature	Skip-gram	CBOW
Input	Target word	Context words
Output	Predict context words	Predict target word
Efficiency	Works well with small data	Faster to train on large data
Captures	Rare words better	Common words better

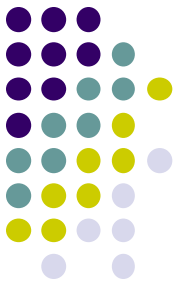
- **Benefits and limitations of CBOW**

- Computationally efficient compared to Skip-gram since it require fewer predictions. Why?
- Smooth by averaging context words
- Context window size may affect the quality of the embeddings.
- Bias toward frequent words

- **Benefits and limitations of Skip-gram**

- Works better in practice and when capturing fine-grained semantic relationships between rare words,
 - useful when accuracy matters

Evaluating Embeddings

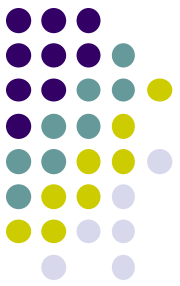


- **Extrinsic evaluation**

- Let's plug these word embeddings into a real NLP system and see whether this improves performance.
 - Could **take a long time** but still the most important evaluation metric

- **Intrinsic evaluation**

- Evaluate on specific/intermediate subtasks
 - Word similarity, word analogy, clustering of words, dimensionality reduction, outlier detection, bias testing, lexical substitution, etc.
 - Fast to compute
- Compare to human scores on word similarity tasks:
 - WordSim-353 (Finkelstein et al., 2002)
 - SimLex-999 (Hill et al., 2015)
 - Stanford Contextual Word Similarity dataset (Huang et al., 2012)
- Not clear if it really helps the downstream tasks
 - Example downstream tasks: sentiment analysis, machine translation, etc.



Intrinsic Evaluation: Word Similarity

- The dataset **wordsim353** has 353 pairs of words with **human judgement**
<http://www.cs.technion.ac.il/~gabr/resources/data/wordsim353/>

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Cosine similarity:

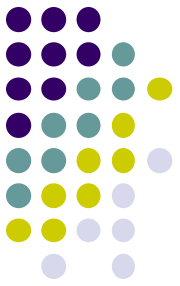
$$\cos(\mathbf{u}_i, \mathbf{u}_j) = \frac{\mathbf{u}_i \cdot \mathbf{u}_j}{\|\mathbf{u}_i\|_2 \times \|\mathbf{u}_j\|_2}$$

Metric: Spearman rank correlation (Larger is better)

MC (Miller-Charles semantic similarity dataset)
RG (RG-65 dataset for association)
SCWS (Sentence similarity dataset)
RW (Relatedness dataset for semantic relatedness)

Model	Size	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	<u>72.7</u>	75.1	56.5	37.0
CBOW [†]	6B	57.2	65.6	68.2	57.0	32.5
SG [†]	6B	62.8	65.2	69.7	<u>58.1</u>	37.2
GloVe	6B	<u>65.8</u>	<u>72.7</u>	<u>77.8</u>	53.9	<u>38.1</u>
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	<u>75.9</u>	<u>83.6</u>	<u>82.9</u>	<u>59.6</u>	<u>47.8</u>
CBOW*	100B	68.4	79.6	75.4	59.4	45.5

Advantages and Limitations of Word2Vec

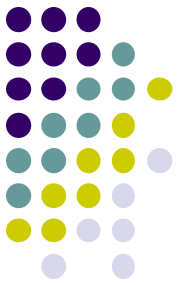


- **Advantages**

- Capturing semantic relationships into dense vectors
- Efficient and scalable (using negative sampling and hierarchical Softmax)
- Pre-trained embeddings that allow transfer learning
- Word analogy and vector arithmetic operations

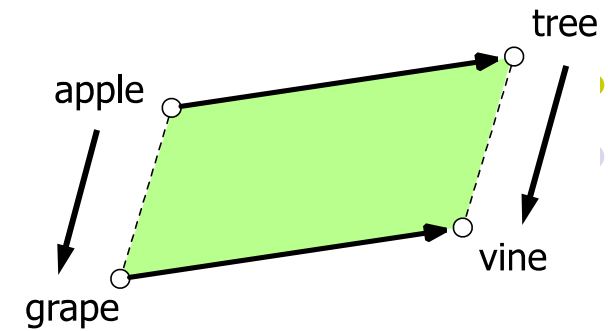
- **Limitations**

- Context-independent
 - Static and single embedding per word, which does not account for polysemy (words with multiple meaning, e.g., “bank”)
- Limited representation of rare words or out-of-vocabulary words
- No sense disambiguation
- Requires large amount of data
 - 10k words and 300 dim embedding gives a large parameter space to learn. 10K words is minimal for many real applications - slow to train.
- Sensitive to hyperparameters (e.g., size of window, dimension)
- Lack of specificity due to unsupervised learning
 - General purpose, not incorporating labeled data from specific tasks
- Bias in word embeddings



Anallogical Reasoning with Embeddings

Analogy Reasoning



- **What is analogical reasoning?**

- Drawing a **conclusion** based on its **relational similarities** to something else that is already known
 - Compared to logical reasoning: deduction, induction, and abduction.

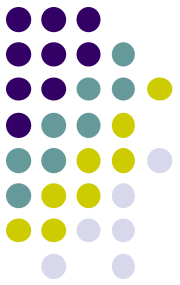
- **Parallelogram model** (Rumelhart and Abrahamson 1973)

- A simple analogy problem of the form $a:a^*:b:b^*$
 - $a^* - a \approx b^* - b$
- To solve “what is b^* ?”, add a^* to get b^* , then $b^* = b + (a^* - a) = a^* - a + b$
 - For example, to solve: “apple:tree::grape:?”
 - Add **tree**, so that **apple** to **grape** to get **vine** (now we can reason “grape vine”)

- **Solving analogies by the parallelogram method**

- Embeddings can capture this parallelogram model of reasoning.
- With both sparse and dense embeddings (Turney et al., 2005, Mikolov et al. 2013b)
 - **King – man + woman** is close to **Queen** (from King:man::Queen:woman).
 - **Paris – France + Italy** is close to **Rome** (from Paris:France::Rome:Italy)

Distributional Semantics in Vector Space



Example: **Analogical reasoning** that captures relational meaning

$\text{vector}(\text{king}) - \text{vector}(\text{man}) + \text{vector}(\text{woman}) \approx \text{vector}(\text{queen})$

$\text{vector}(\text{Paris}) - \text{vector}(\text{France}) + \text{vector}(\text{Italy}) \approx \text{vector}(\text{Rome})$

Test for linear relationships, examined by Mikolov et al. (2014)

a:b :: c:?



$$d = \arg \max_x \frac{(w_b - w_a + w_c)^T w_x}{\|w_b - w_a + w_c\|}$$

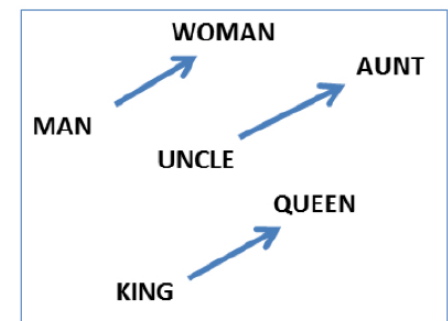
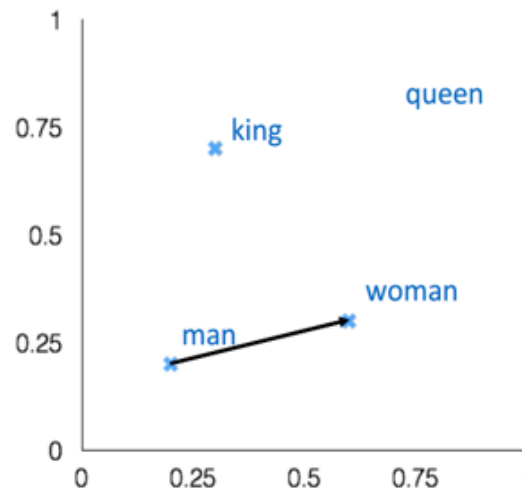
man:woman :: king:?

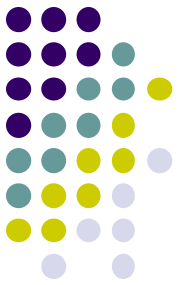
+ king [0.30 0.70]

- man [0.20 0.20]

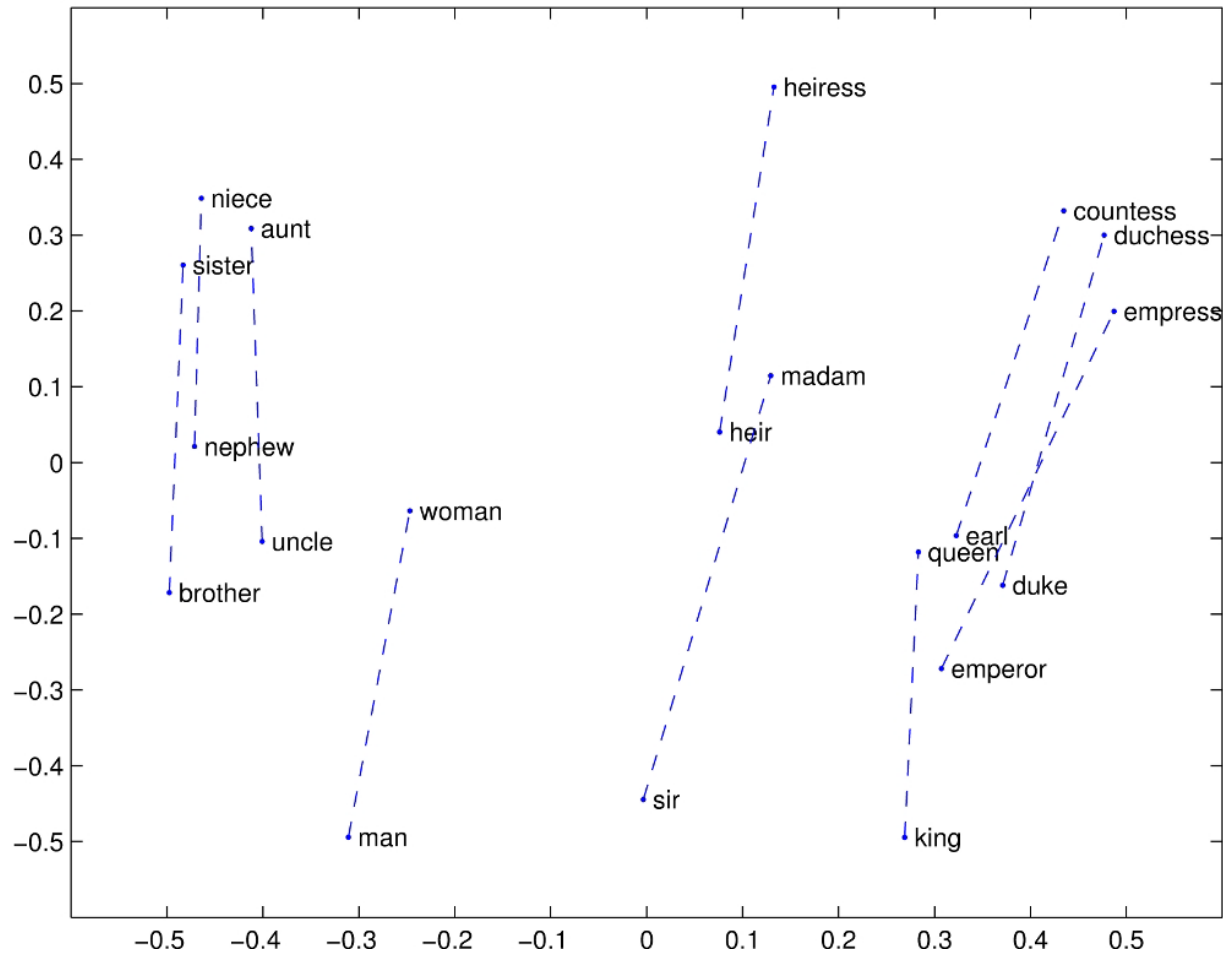
+ woman [0.60 0.30]

queen [0.70 0.80]

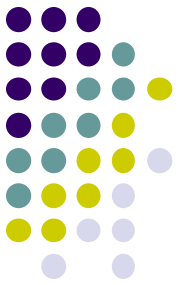




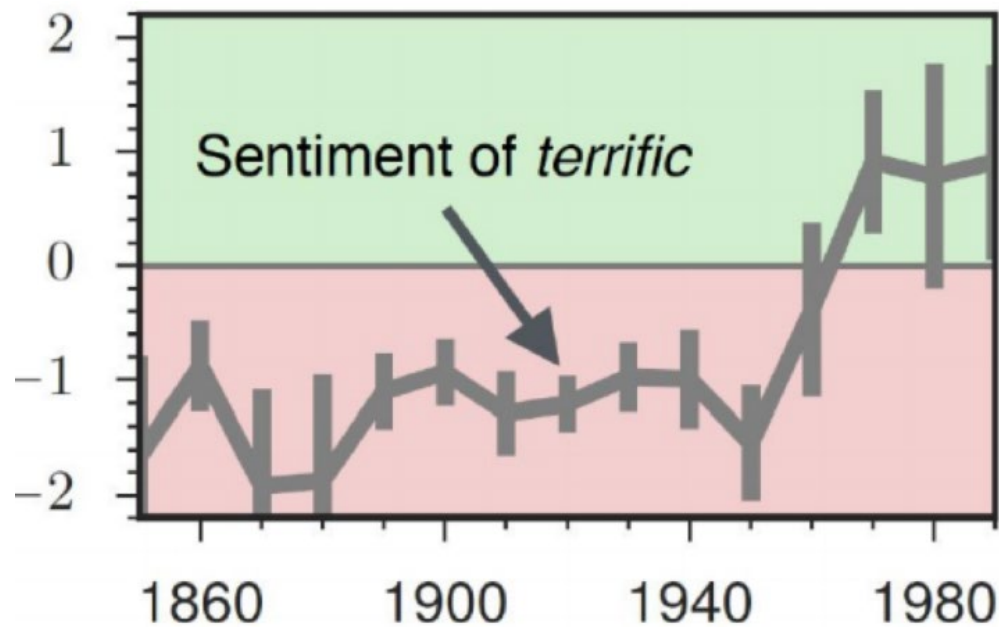
Example: Analogy in Embeddings



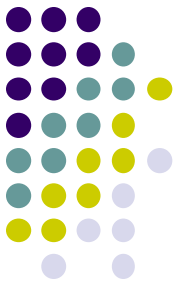
Structure in GloVe embeddings space



The Evolution of Sentiment Words

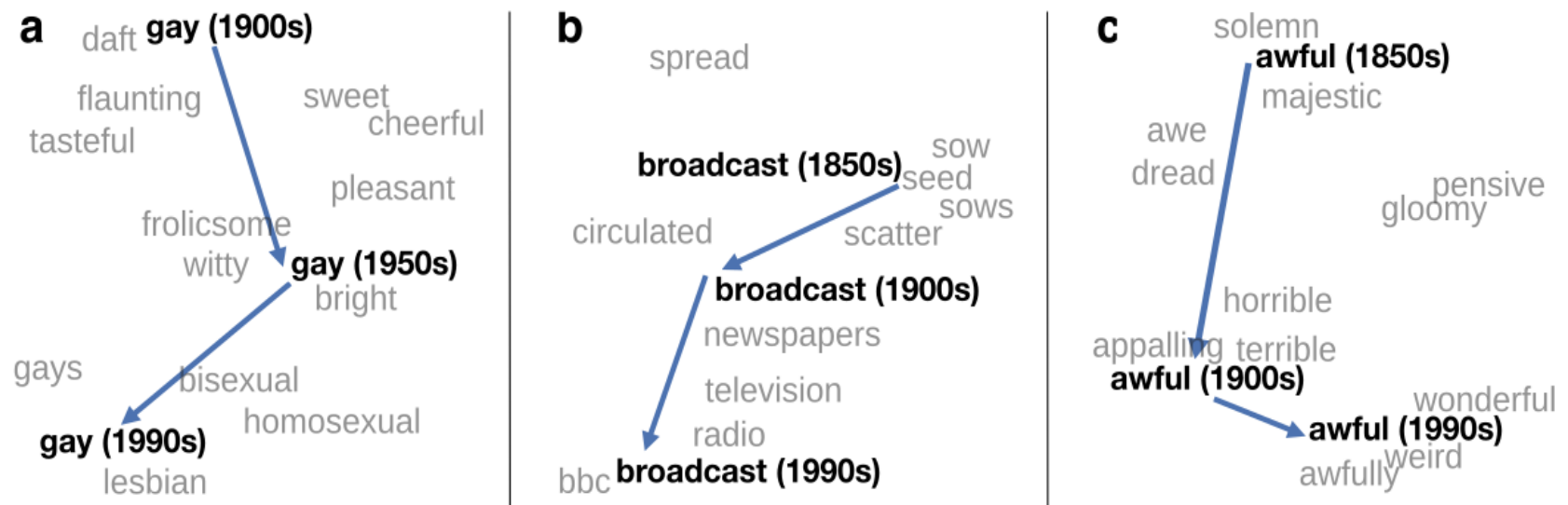


Embeddings as a Window onto Historical Semantics



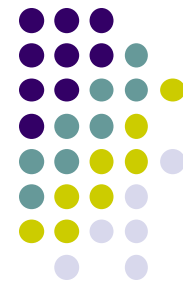
Embeddings on different decades of **historical text** to see **meanings shift** – **language changes**

~30 million books, 1850-1990, Google Books data
Projected ~300 dimensions down to 2D



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Caveats with the Parallelogram Method

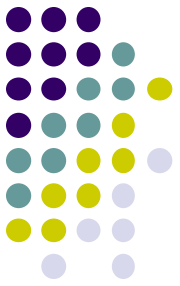


- It only seems to work for **frequent words**, **small distances** and **certain relations** (relating countries to capitals, or parts of speech), **but not others**

(Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)

- **Understanding analogy is an open area of research!**

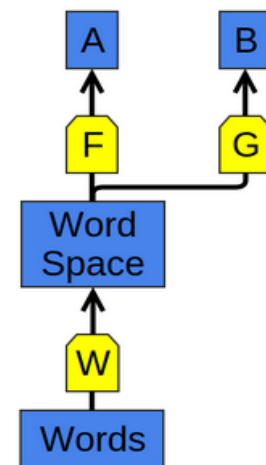
(Peterson et al. 2020)



Applications of Word Embeddings

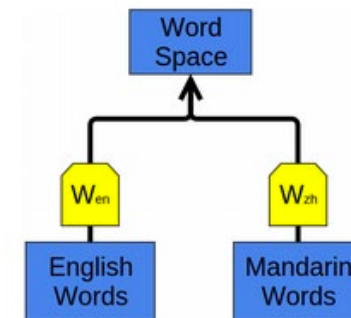
Pretraining and Transfer Learning

- The use of word representations... has become a key “**secret sauce**” for the **success of many NLP systems** in recent years, across tasks including named entity recognition, part-of-speech tagging, parsing, semantic role labeling, etc. (Luong *et al.* 2013)
- **Pretraining and transfer learning**
 - Learning a good representation on a **task A**, then using it on a **task B** is one of the major tricks in the **Deep Learning toolbox**.
 - An example of transfer learning
 - Can allow the representation to **learn from more than one kind of data**.



W and F learn to perform task A. Later, G can learn to perform B based on W.

Multimodal Learning



- **Multimodal learning**

- Training models on data from different modalities, such as text, images, audio, or even sensor data, to create a unified representation that captures relevant information from the different types of data
 - Learning to map multiple kinds of data into a single representation

- **Example: Learning from different languages**

- English and Mandarin Chinese word-embedding (Socher *et al.* 2013a)
 - Words that are known as close translations should be close together.
 - Words we *didn't know* were translations end up close together!
- Structures of two languages get pulled into alignment.

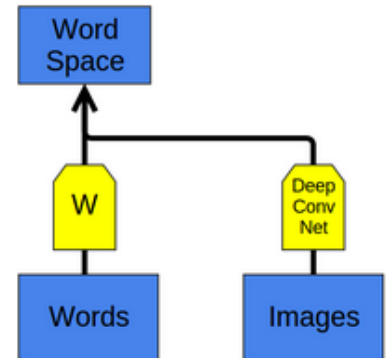
- **Example: Audio-Visual**

- The model maps both audio and visual lip movement data into a shared representation, improving its ability to recognize spoken words.

Multimodal Learning

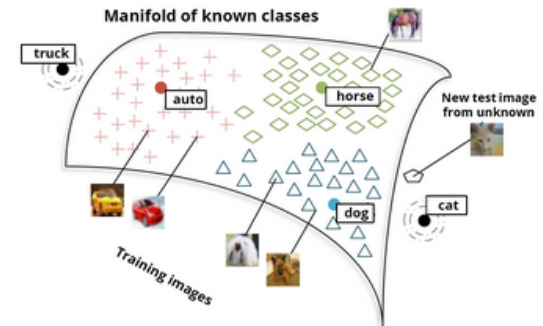
- Example: **Text-Image**

- The model learns to map both images and text into a shared latent space, allowing it to perform tasks such as image-caption matching or zero-shot image classification.



- Example: **New class mapping**

- New classes map near similar existing classes.
 - e.g., if 'cat' is unknown, cat images map near dog.



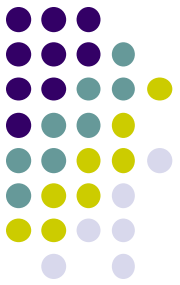
(Socher *et al.* (2013b))

- Example: **Sensor fusion**

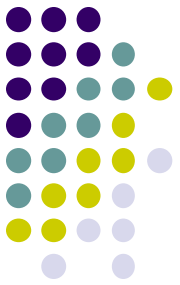
- Multiple sensor inputs (radar, camera, LiDAR) are combined into a single representation to make decisions about the environment in robotics or autonomous driving.

- **Many more applications**

- The potential is enormous!



Some Improvement Ideas for Word2Vec



Word Pairs and Phrases

- **Idea**

- Treat common word pairs or phrases as single “words.”
 - E.g., “Boston Globe” (newspaper) is different from “Boston” and “Globe” separately.

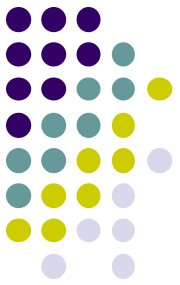
- **Word phrases**

- Make **phrases** out of words which occur together often relative to the number of individual occurrences.
- Prefer **phrases** made of **infrequent words** in order to avoid making phrases out of common words like “and the” or “this is”.
- **Results**
 - Led to 3 million “words” trained on 100 billion words from a Google News dataset

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Word Pairs

- Why not capture co-occurrence counts directly?
 - Building a co-occurrence matrix
 - Example: window-based co-occurrence matrix
 - Window size 1 (more common: 5-10) or Symmetric (left or right context is irrelevant)
 - Example corpus, e.g., co-occurrence count “I” and “like” is 2
 - I like deep learning
 - I like NLP
 - Simple count co-occurrence vectors
 - Vector size with vocabulary: **high dimensional** (storage and sparsity issues)
- Low-dimensional vectors
 - Store “**most**” important information in a fixed, small dimensional dense vector (25-100 dimensions similar to word2vec)
 - How to reduce the dimensions? **SVD** or **matrix factorization** (GloVe)
- **Pros/cons**
 - Increases vocabulary size but decreases training expense



Subsample Frequent Words

- **Idea**

- Reduce the frequent words.
 - To decrease the number of training examples
 - The probability that we cut the word is related to the word's frequency.
 - More common words are cut more, e.g., removing some occurrences of “the”.
- Uncommon words (e.g., anything $< 0.26\%$ of total words) are kept.

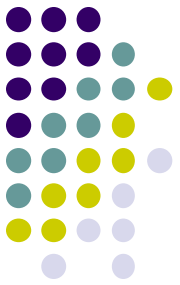
- **Method**

- For each word, cut the word with probability related to the word's frequency.

- **Benefits**

- Reducing the frequent words but keeping rare words
 - If we have a window size of 10 and we remove a specific instance of “the” from our text,
 - as we train on the remaining words, “the” will not appear in any of their context windows.

Subword Representation



- **Idea**

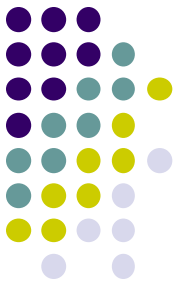
- Subword representation: Skiing = {^skilling\$, ^ski, ski, kiin, iing, ing\$}
 - Reasoning about structure below the word level (parts of words, subtokens, characters, bytes), originally used for machine translation

- **Method**

- Each word is split into a sequence of known subwords
- Reason about subwords
 - Algorithms such as the Byte-Pair encoding, WordPiece, etc
 - A simple and effective strategy for defining a subword vocabulary

- **Benefits**

- Better handling of rare and OOV words
- Capturing morphological structure
- Improved generalization across words
- Smoother representation, handling spelling variants and errors
- Reduced vocabulary size and efficient use of data
- Perform better in low-resource settings (where limited data)



Subword Tokenization

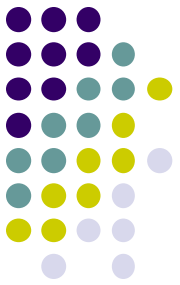
- **Three common algorithms:**

- WordPiece (Schuster and Nakajima, 2012)
 - Subword merges are based on the likelihood (MLE) of the tokens.
 - More context aware but slower and more complex, used in BERT
- Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
 - Subword merges are based on frequency of the tokens.
 - **Simpler and faster, used in GPT**
- Unigram language modeling tokenization (Kudo, 2018)

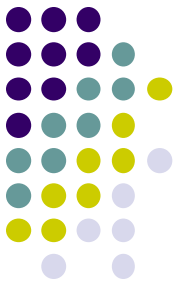
- **All have 2 parts:**

- **A token learner** that takes a raw training corpus and **induces a vocabulary** (a set of tokens).
- **A token segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary **learned through the token learner**.

The Byte-Pair Encoding (BPE) Token Learner Algorithm



- Let vocabulary be the set of all individual characters
= {A, B, C, D,..., a, b, c, d....}
- **Repeat:**
 - Choose the **two symbols** that are **most frequently adjacent** in the **training corpus** (say 'A', 'B')
 - Add a new merged symbol 'AB' to the vocabulary
 - Replace every adjacent 'A' 'B' in the corpus with 'AB'.
- **Until** k merges have been done.



Example: BPE Token Learner

Original corpus:

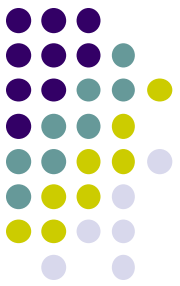
“low low low low low lowest lowest newer newer newer newer newer newer
wider wider wider new new”

Add end-of-word tokens “ ”, resulting in this vocabulary:

vocabulary

—, d, e, i, l, n, o, r, s, t, w

- Most subword algorithms are **run inside words** (not merging across word boundaries), so the input corpus is first white-space separated to give the characters of a word plus an **end-of-word token** like “ ”.



Example: BPE Token Learner

count end-of-word symbol

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

The algorithm first **counts** all pairs of **adjacent symbols**, then **merge** the **most frequent pair** and treat the merged one as **one symbol** and **count again**.

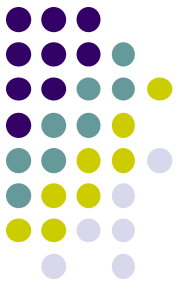
Merge **e** and **r** to **er** as its pair **e** and **r** occurs a total of **9** and **add** it to vocabulary list.

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er



Example: BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, e r

Next most frequent pair “er” and “_” are merged to “er_”, then added to the vocabulary list.

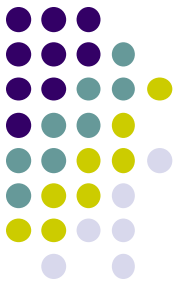
Merge er and _ to er_

corpus

5 l o w _
2 l o w e s t _
6 n e w e r_
3 w i d e r_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, e r, er



Example: BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

Next “n” and “e” (total count of 8) get merged to “ne”
added to the **vocabulary list**.

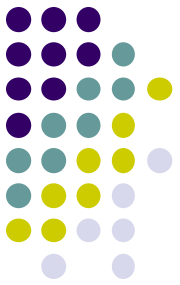
Merge n and e to ne

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne



Example: BPE

Original corpus:

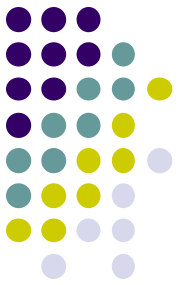
“low low low low low lowest lowest newer newer newer newer newer wider wider wider new new”

If we continue, the next merges are:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

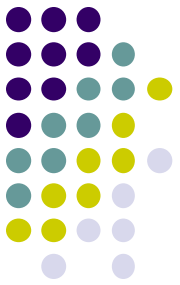
By the end, the character sequence, “n” “e” “w” “e” “r” “_” would be tokenized as a full word “**newer_**”.

But the characters of a new (unknown) word like “l” “o” “w” and “e” “r” “_” would be merged into the two tokens “**low**” and “**er_**” (as **no** “**lower**” is in the **original corpus**).



BPE Token Segmenter Algorithm

- Once **learned** the **vocabularies**, the **token segmenter** is used to tokenize a **test sentence**.
 - **On the test data**, **run each merge** learned from the training data greedily in the order we learned them (**test frequencies don't play a role**):
 - (a) Segment each test sentence word into characters
 - (b) Replace every instance of “e “r” with “er” in the test corpus
 - (c) Replace every instance of “er” “_” with “er_” and so on.
 - **Results of Token Segmenter:**
 - Test set “n” “e” “w” “e” “r” “_” would be tokenized as a full word “**newer_**”
 - Test set “l” “o” “w” “e” “r” “_” would be two tokens: “**low**” “**er_**”
- **Properties of BPE tokens**
 - Usually include frequent words and frequent subwords, which are often morphemes like “**-est**” or “**-er**”
 - A **morpheme** is the smallest meaning-bearing unit of a language
 - The word “**Unlikeliest**” has 3 morphemes “**un-**”, “**likely**”, and “**-est**”



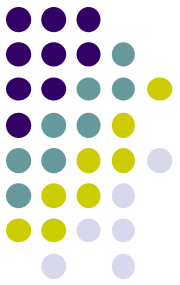
Word Structure and Subword Models

	word		vocab mapping	embedding
Common words	hat	→	hat	
	learn	→	learn	
Variations	taaaaasty	→	taa## aaa## sty	
misspellings	laern	→	la## ern##	
novel items	Transformerify	→	Transformer## ify	

- **Some issues of subword representation**

- Common words end up being a part of the subword vocabulary, while rarer words are split into (sometimes intuitive, sometimes not) components.
- In the worst case, words are split into as many subwords as they have characters.

- **This is a dominant modern NLP method!**



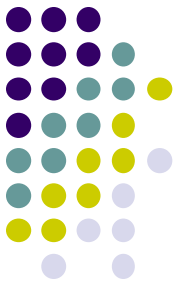
Extension of Word2vec and Variants

- **Extension of Word2vec embeddings**

- **Word2vec** (Mikolov et al., 2013, <http://code.google.com/archive/p/word2vec/>)
- **Global Vectors (GloVe)** (Pennington et al., 2014, <http://nlp.Stanford.edu/projects/glove>)
 - Global contextual information (based on global co-occurrence statistics)
 - Dimensionality reduction
 - Smooth embedding space (weighted least square objective function)
- **FastText** (Bojanowski et al., 2017 @Facebook AI research, <https://fasttext.cc/>)
 - Enriching word vectors with Subword Information
 - Improved handling of rare and OOV words
 - Multilingual capabilities
- **ELMO** (Allen Institute for AI@ UofWashington, 2018, <https://allennlp.org/elmo>)
 - Contextualized token embeddings (sequence modeling using LSTM)

- **Many more variants**

- Multilingual embeddings (canonical correlation analysis)
- Multi-sense embeddings
- Syntactic embeddings
- Etc.



FastText Evaluation

- **Intrinsic evaluation**

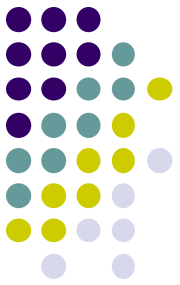
word1	word2	similarity (humans)
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

similarity (embeddings)
1.1
0.5
0.3
1.7
0.98
0.3

Spearman's rho (human ranks, model ranks)

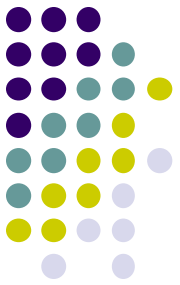
- **Multilingual**

- Arabic, German, Spanish, French, Romanian, Russian



Pretrained Word Embeddings

- Word2vec (Mikolov et al. 2013)
 - <https://code.google.com/archive/p/word2vec/>
- GloVe (Global Vectors, Pennington et al., 2014)
 - <https://nlp.stanford.edu/projects/glove/>
- FastText (Facebook)
 - <https://fasttext.cc/>
- Etc.



Sentence Embeddings

- **Issues of word embeddings**

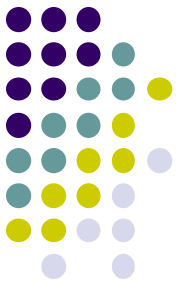
- Unable to handle semantic similarity, retrieval, or classification that requires the meaning of a whole sentence.

- **Why sentence embeddings?**

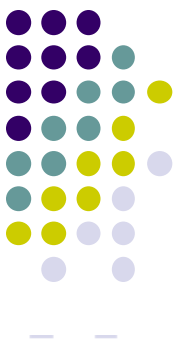
- Semantic search: find similar sentences/documents
- Text classification: use sentence embeddings as input features
- Clustering/topic modeling: group similar sentences or paragraphs
- Question answering/retrieval-augmented generation:
 - Match questions to relevant text passages

- **Some methods**

- Averaging word embeddings
 - Simple and fast but loses word order and nuanced meaning
- Sentence transformer
 - Sentence-BERT, Universal Sentence Encoder (USE)



Bias in Embeddings



Identifying Bias in Embeddings

- Identify “**she**” and “**he**” axis in word vector, **project** words onto **this axis**.

1. homemaker
4. librarian
7. nanny
10. housekeeper

Extreme *she* occupations

2. nurse
5. socialite
8. bookkeeper
11. interior designer
3. receptionist
6. hairdresser
9. stylist
12. guidance counselor

- Apply **nearest neighbor** ($a - b + c$)
 - The vector arithmetic designed to reveal implicit relationships between word vectors (**vector offset**)

1. maestro
4. philosopher
7. financier
10. magician

Extreme *he* occupations

2. skipper
5. captain
8. warrior
11. fighter pilot
3. protege
6. architect
9. broadcaster
12. boss

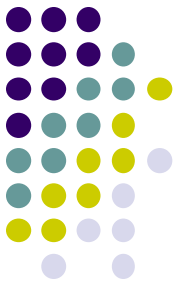
Bolukbasi et al., 2016

- Examples**

- If a is “**king**”, b is “**man**”, and c is “**woman**”, the vector $a - b + c \Rightarrow$ “**queen**”.
- $(a - b)$ captures the gendered relationship “male-to-female royalty”, this offset is transferred to c .
- Similarly, **doctor** – **man** + **woman** $\Rightarrow$ “**nurse**” (by the result of nearest neighbor)

Racial Analogies	
black $\rightarrow$ homeless	caucasian $\rightarrow$ servicemen
caucasian $\rightarrow$ hillbilly	asian $\rightarrow$ suburban
asian $\rightarrow$ laborer	black $\rightarrow$ landowner
Religious Analogies	
jew $\rightarrow$ greedy	muslim $\rightarrow$ powerless
christian $\rightarrow$ familial	muslim $\rightarrow$ warzone
muslim $\rightarrow$ uneducated	christian $\rightarrow$ intellectually

Manzini et al., 2019



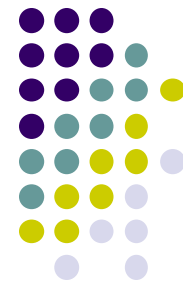
Embeddings Reflect Cultural Bias

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

- Ask “Paris: France :: Tokyo : x”
 - x = Japan
- Ask “father: doctor :: mother : x”
 - x = nurse
- Ask “man: computer programmer :: woman : x”
 - x = homemaker

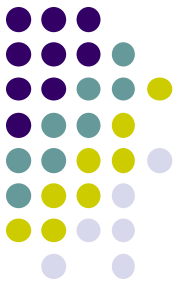
Algorithms that use embeddings as part of hiring searches for programmers, might lead to bias in hiring.

Historical Embedding as a Tool to Study Cultural Biases



Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. *Proceedings of the National Academy of Sciences* 115(16), E3635–E3644.

- The method used in their study
 - Compute a **gender** or **ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities.
- **Study results**
 - **Embeddings** for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical*) are **biased toward men**, a bias slowly decreasing 1960-1990.
 - **Embeddings** for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were **biased toward Asians** in the 1930s, **bias decreasing over the 20th century**.
 - These match the results of old surveys done in the 1930s.



Biased NLP Technologies

- Bias in Word Embeddings (Bolukbasi et al., 2017)
- Bias in Language Identification (Jurgens et al., 2017)
- Bias in Visual Semantic Role Labeling (Zhao et al., 2017)
- Bias in Natural Language Inference (Rudinger et al., 2017)
- Bias in Coreference Resolution (Zhao et al., 2018)
- Bias in Automated Essay Scoring (Amorim et al., 2018)
- Etc.

Sources of Biases



- Humans are the “natural” in NLP
 - Who generate the data and select the data?
 - Natural language data and annotations will reflect social/cognitive biases.
 - Bias in narratives creates social and racial stereotypes.
 - Biases in data sampling
- Cognitive biases in cognitive science and psychology
 - **Stereotypes inevitably form** because of the **innate tendency** of the human mind to categorize the world **to simplify processing**.
 - Social stereotypes are not necessarily bad, but still have **negative effect**.
 - **Implicit biases** are distressingly **pervasive**, operate largely **unconsciously**, and can automatically influence the ways in which we see and treat others, even when we are determined to be fair and objective.
- **ML algorithms**
 - Will **replicate biases** in their training data and **amplify** them through training – **inductive bias** in word embeddings.
 - E.g., Man – Woman $\approx$ Computer Programmer – Homemaker

Bias in Data and Sampling

- **Self-selection bias**

- Who decides to post reviews on Yelp and why?
- Who posts on Twitter and why?

- **Reporting bias**

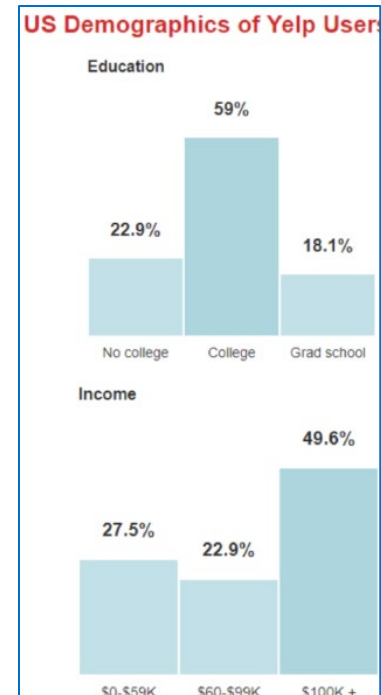
- People do not necessarily talk about things in the world in proportion to their empirical distributions (Gordon and Van Durme 2013).

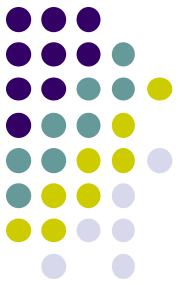
- **Motivational bias**

- Paid vs unpaid vs implicit participants

- **Community/Dialect/Socioeconomic biases**

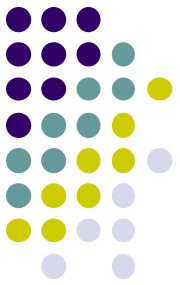
- What linguistic communities are over- or under-represented?
 - About 1.35 billion people worldwide speak English.
- Most employers use off-the-shelf language identification systems which are highly accurate (Jurgens et al., 2017).
- Leads to community-specific model performance (Jorgensen et al., 2015)





Debiasing Method: Hard Debiasing

- **The goal is to remove bias.**
- **Steps**
 1. Identify the bias subspace
 - Use word pairs that define the bias e.g., (“he”, “she”) for gender bias.
 - Perform PCA on these pairs to find the direction in the vector space that corresponds to the bias (called “bias subspace”).
 2. Neutralization
 - For words that should be neutral, e.g., (“doctor”, “teacher”), remove the component of the word vector along the bias subspace, making “doctor” equidistance to “he” and “she”.
 3. Equalization
 - For words that should be equalized, e.g., (“king”, “queen”), adjust the word vectors so that both are equally far from any neutral terms like “doctor” and lie symmetrically along the bias axis.

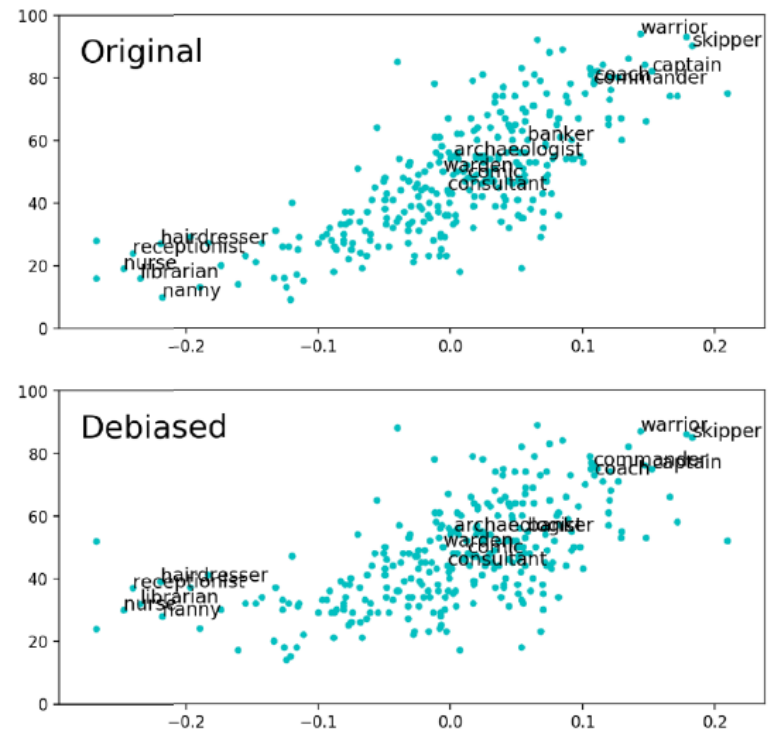


Debiasing Method: Soft Debiasing

- **The goal is to reduce bias.**
- **Soft debiasing**
 - Identify the bias subspace.
 - Partially reduce bias (instead of completely removing the bias).
 - To reduce its influence by regularizing word vectors for the embeddings
 - To retain some degree of relevant information like gender when needed for meaningful tasks, but reduces the unwanted, stereotypical bias
- **Other methods**
 - Counterfactual data augmentation, adversarial debiasing, bias regularization, word embedding association test, contextualized embedding debiasing, fair representation learning, etc.

Difficulty of Debiasing

- **Not that effective**...and the male and female words are still clustered together.
- **Bias pervades** the word embedding space and isn't just a local property of a few words.



(a) The plots for **HARD-DEBIASED** embedding, before (top) and after (bottom) debiasing.

Gonen and Goldberg (2019)